

**UNIVERSITY OF CALIFORNIA
SANTA CRUZ**

SATURN: ZERO-CONFIGURATION AI SERVICE DISCOVERY

A thesis submitted in partial satisfaction
of the requirements for the degree of

MASTER OF SCIENCE

in

COMPUTER SCIENCE

by

Joseph Perrello

March 2026

The Thesis of Joseph Perrello
is approved:

Adam Smith, Chair

Ram Sundara Raman

Peter Biehl

Vice Provost and Dean of Graduate

Studies

Copyright © by
Joseph Perrello
2026

Contents

List of Figures	viii
List of Tables	ix
Abstract	x
Generative AI Acknowledgement	xii
1 Introduction	1
2 Background	5
2.1 Service Discovery as a Problem	6
2.2 Multicast DNS and DNS-Based Service Discovery	7
2.2.1 TXT Records	8
2.2.2 Platform Implementations	8
2.2.3 Privacy Considerations	9
2.3 Alternative Service Discovery Protocols	9
2.4 AI Applications That Consume Text Completions	10
2.5 DHCP as Analogy and Contrast	12
2.6 The Gap Saturn Addresses	13
3 Design	14

3.1	Design Goals	14
3.2	Audiences	15
3.3	Concepts	16
3.3.1	Endpoints	16
3.3.2	Beacons	17
3.3.3	Priorities	18
3.3.4	Ephemeral Keys	18
3.4	Protocol Specification	19
3.4.1	Service Type	19
3.4.2	DNS Record Structure	20
3.4.3	TXT Record Schema	20
3.4.4	Discovery Flow	21
3.5	Architecture Decisions	22
3.5.1	Beacon, Not Proxy	22
3.5.2	OpenAI API as Interface Standard	23
3.5.3	Priority as Policy	24
3.6	Security	24
3.6.1	Threat Model 1: Corporate Data Collection	25
3.6.2	Threat Model 2: Untrusted Administrator	25
3.6.3	Broadcast Exposure	26
3.7	Summary	27
4	Implementation	28
4.1	Shared Foundation	29
4.2	VLC Extension: AI in a Non-AI-Native Application	30
4.2.1	Architecture	30

4.2.2	Technology Choices	31
4.2.3	Walkthrough: From Launch to Chat	31
4.2.4	What the VLC Extension Proves	32
4.3	Saturn Router: AI at the Network Infrastructure Layer	33
4.3.1	Architecture	33
4.3.2	Technology Choices	34
4.3.3	OpenWRT Integration	35
4.3.4	Walkthrough: Deploying Saturn on a Router	36
4.3.5	What the Router Proves	37
4.4	OpenCode: Saturn in an AI Coding Agent	38
4.4.1	Integration Approach	38
4.4.2	Dynamic Registration	39
4.4.3	Integration Challenges	39
4.4.4	What OpenCode Proves	40
4.5	Summary of Implementations	41
5	Evaluation	43
5.1	Claim 1: Zero-Configuration AI Provisioning Is Feasible	44
5.1.1	Methodology	44
5.1.2	Results	44
5.1.3	Assessment	46
5.2	Claim 2: Reduced Configuration Effort	46
5.2.1	Methodology	46
5.2.2	Results	47
5.2.3	What Drives the Reduction	48
5.2.4	Assessment	49

5.3	Claim 3: Security Trade-offs Are Known and Addressable	49
5.3.1	Methodology	50
5.3.2	Results	50
5.3.3	Assessment	52
5.4	Cross-Cutting Evaluation: Nielsen Heuristic Analysis	53
5.4.1	Methodology	53
5.4.2	Strengths	53
5.4.3	Trade-offs	54
5.4.4	Limitations	55
5.5	Threats to Validity	55
6	Discussion	58
6.1	Implications for AI Access	58
6.2	Relationship to Prior Work	60
6.3	Limitations	61
6.4	Future Work	63
6.5	Significance	64
7	Conclusion	65
A	Saturn Protocol Specification	67
A.1	Service Type	67
A.2	TXT Record Schema	68
A.3	Required Endpoint Paths	70
A.4	Discovery Sequence	71
A.5	Beacon Format and Credential Distribution	72
A.6	Priority Scheme	73

A.7	Ephemeral Key Lifecycle	74
A.8	Conformance Requirements	74
B	Generative AI in Thesis Development	76
B.1	Design Fictions	77
B.2	Documentation Website	78
B.3	Integrations into Various Applications	79
B.4	Serve Role as a “Robotic” Advisor	79
B.5	Moons	80
B.6	Finding Academic Papers	84
B.7	AI Contribution in the Evaluation Section	84
B.8	Academic Writing Skill	85
B.9	AI Detection Ralph Loop	87
B.10	Structured Arguments Skill	88

List of Figures

List of Tables

3.1	Saturn TXT record fields. Fields marked required must appear in every beacon advertisement.	21
4.1	Three Saturn implementations demonstrating progressive scope expansion. Each uses a different mDNS library, supporting argument A10 : seven consumers across three languages and four mDNS libraries inter-operate on <code>_saturn._tcp.local</code> . with no bridging layer.	41
5.1	Cognitive Walkthrough Step Counts by Persona	48

Abstract

Saturn: Zero-Configuration AI Service Discovery

Joseph Perrello

Generative AI services gate access behind per-user subscriptions, API keys, and manual endpoint configuration—barriers that scale with the number of users and applications on a network. Saturn provisions AI the way printers and speakers already provision themselves: through Multicast DNS (mDNS) and DNS-based Service Discovery (DNS-SD), protocols that ship on every major operating system. An administrator registers AI endpoints under the service type `_saturn._tcp.local.`; every device on the network segment discovers them without accounts, credentials, or configuration files.

This thesis specifies the Saturn protocol—a service type, a TXT record schema carrying endpoint URLs, API types, priorities, and ephemeral credentials—and presents its architecture of beacons that announce but never proxy traffic, preserving user privacy by keeping the announcer off the data path. Three implementations demonstrate progressive scope: a VLC media player extension that brings AI to a non-AI-native application, a router firmware integration that provisions AI at the network infrastructure layer on a \$30 MIPS32 device, and an AI coding agent fork that handles streaming tool calls and multi-turn conversation. Seven consumers across three languages and four independent mDNS libraries interoperate on the protocol with no shared discovery code, demonstrating that the specification—not a shared library—carries the design.

A cognitive walkthrough across three personas (administrator, developer, end user) measures a 53% reduction in total configuration steps compared to traditional per-user API key distribution, with developer effort dropping 79% and end-user configuration

eliminated entirely. At modest scale (ten developers, one hundred users), the reduction reaches 94%. Cloud-facing deployments rotate ephemeral API keys every five minutes; a captured credential expires before an attacker can act on it, sidestepping the static key leakage class that prior work found affects over 100,000 public repositories.

Saturn’s broadcast model inherits known mDNS privacy risks—passive service enumeration, metadata leakage to all devices on the subnet—and enterprise WiFi access-point isolation blocks multicast entirely, confining viable deployment to home, office, and lab networks. These trade-offs are grounded in existing mDNS security literature and addressable through ephemeral credentials, spending limits, and privacy-preserving protocol extensions without destroying the zero-configuration property.

Generative AI Acknowledgement

All of the text in this thesis excluding Appendix B and *this* acknowledgement is be the output of Anthropic’s Claude series of LLMs (Opus 4.6, Sonnet 4.5, and Haiku 4.5)¹ that I (Joey Perrello) obtained while working with Anthropic’s coding agent Claude Code.² This acknowledgement exists because I (and my advisors, Adam and Ram) believe there should be full disclosure of generative AI when used in the creation of academic work, as some readers may be morally opposed to generative AI. Previous academic work went uncredited into the training of the models, so it is the responsibility of academics to attempt to show respect to those who contributed before by acknowledging use of LLMs. My advisors recommended this thesis’ use of generative AI under the conditions that there was a *human-written* acknowledgement and an appendix entry detailing how the system works. The resulting system is an agent-assisted research flow that is systematic and comprehensive, and an integral aspect of this thesis’ creation. I originally intended Saturn, the protocol discussed in this thesis, to be a Master’s project, where the guidelines for completion were more relaxed than a Master’s thesis. I decided at the beginning of the academic quarter to transition to a thesis. During this time I became inspired by the agent driven development work of Geoffrey Huntley (of Ralph loops), Dexter Horthy (of 12-factor agents), and Steve Yegge (of

¹<https://www.anthropic.com/system-cards>

²<https://code.claude.com/docs/en/overview>

Gas Town). This inspiration spawned the idea of an academic research assistant with the same knowledge and comprehension of Saturn as my own. However, this system ended up requiring the same level of attention and focus as Saturn, with similar levels of complexity. The system and all use cases of LLMs in this thesis is described in Appendix B.

Chapter 1

Introduction

In 1954, Lewis Strauss predicted that nuclear energy would make electricity “too cheap to meter” [38]. The prediction failed for energy, but its logic — that a resource becomes transformative once its marginal cost approaches zero — resurfaced seventy years later in a different domain. Large language models now generate text, translate languages, summarize documents, and write code at a per-query cost measured in fractions of a cent [31]. The computational cost of a single inference has fallen fast enough that organizations routinely absorb it for millions of users. AI, in a narrow but meaningful sense, is approaching “too cheap to meter.”

The access model has not kept pace with the cost curve. Today’s dominant pattern for consuming AI services requires each user to create an account with a provider, supply payment credentials, obtain an API key, and configure that key into every application that needs it [40]. A student who wants AI-assisted code completion, AI-powered note-taking, and AI-driven transcription must manage three separate subscriptions, three API keys, and three billing relationships — or pay \$20–100 per month for a bundled consumer plan that locks capabilities to a single vendor’s interface [30, 1]. Bassignana et al. [4] demonstrated that this subscription model creates measurable dis-

parities: users from lower socioeconomic backgrounds interact with language technologies less frequently and derive less benefit from them. Gabriel [13] documented the same pattern in education, where students without personal AI subscriptions fall behind peers who can afford them. The cost of inference may be approaching zero, but the cost of *accessing* inference remains a barrier.

This barrier motivated the present work. As a graduate student paying \$100 per month across several AI services, I recognized that many of my peers could not afford the same access. Universities already provision shared resources at the network level: a student who joins eduroam gains access to printers, file shares, and licensed software without creating accounts or entering credentials. The network itself carries the configuration. I asked whether AI could work the same way — whether joining a local network could grant access to AI services with no accounts, no API keys, and no per-user billing.

The answer required a protocol. Existing zero-configuration networking protocols already solve an analogous problem for simpler services. Multicast DNS (mDNS) and DNS-based Service Discovery (DNS-SD), standardized in RFC 6762 [8] and RFC 6763 [7], allow devices to advertise and discover services on a local network without any centralized registry or manual configuration. Guttman [15] established the design philosophy: “enable direct communications between two or more computing devices via IP” with no prior configuration. Apple’s Bonjour [3] and the open-source Avahi [33] project proved this approach viable for printers, file shares, and media servers. Siljanovski et al. [37] showed that mDNS adapts to constrained environments, and Kim and Reeves [20] traced the protocol’s lineage from printer discovery to broader service ecosystems. No one, however, has applied these protocols to AI service provisioning. The gap is specific: no existing system uses zero-configuration network protocols to advertise, discover, and connect to AI service endpoints.

Saturn fills this gap. I define *Saturn* as the process of advertising AI service API endpoints via mDNS packets on a local network, discovering those endpoints through standard DNS-SD queries, and transferring data between client and service using the discovered connection parameters. A *Saturn service* is any AI endpoint advertised via the `_saturn._tcp.local.` service type. Saturn does not proxy traffic or relay inference requests; it broadcasts a *beacon* — a set of mDNS TXT records containing the endpoint URL, authentication credentials, API type, and priority — so that any application on the network can discover and connect to AI services directly. An administrator registers a service once; every device on the network discovers it automatically. The consumer calls `discover()` and receives a working API endpoint. No URLs, no keys, no configuration files.

This thesis advances three claims about Saturn’s viability and implications:

- V1** Zero-configuration network protocols can provision AI services without end-user configuration. Saturn’s implementation across six independent clients, four mDNS libraries, and three programming languages demonstrates that mDNS/DNS-SD can carry AI service metadata with no bridging layer or protocol extensions.
- V2** Network-provisioned AI reduces total configuration effort compared to per-user manual setup. A cognitive walkthrough using the methodology of Wharton et al. [42] shows that Saturn reduces the combined configuration steps across administrators, application developers, and end users by 53%, with application developers experiencing a 79% reduction and end users performing zero configuration steps.
- V3** The security trade-offs of broadcast-based AI discovery are documented in existing literature and can be mitigated without destroying the zero-configuration property. Saturn deliberately trades enterprise-grade verification for zero-configuration

access — a design choice grounded in the mDNS threat models documented by Kaiser and Waldvogel [18] and mitigated through ephemeral credentials with ten-minute expiry windows.

Together, these claims argue that AI access can and should be provisioned at the network level. Saturn demonstrates the technical feasibility of this model, quantifies its usability benefits, and maps the security surface that any future deployment must address.

The remainder of this thesis is organized as follows. Chapter 2 surveys zero-configuration networking, AI-as-a-Service delivery models, and the service discovery protocols that Saturn builds upon. Chapter 3 presents Saturn’s architecture, protocol specification, and security model. Chapter 4 describes six client implementations spanning VLC media player, a portable travel router, and an AI coding agent. Chapter 5 evaluates Saturn against its three claims through protocol analysis, a cognitive walk-through, and a security assessment. Chapter 6 examines deployment implications, the AP isolation constraint that limits institutional adoption, and Saturn’s relationship to the broader AI democratization effort. Chapter 7 summarizes the contributions and identifies directions for future work.

Chapter 2

Background

Saturn broadcasts AI service endpoints over the local network so that any device can discover and use them without configuration. This chapter establishes the technical context a reader needs before evaluating that design. It opens with the problem Saturn addresses: every AI service today requires its own configuration, and no standard protocol announces AI capabilities the way mDNS announces printers. It then covers the protocol Saturn builds on—mDNS and DNS-based Service Discovery—in enough depth to follow the design decisions in Chapter 3. It surveys four alternative service discovery protocols to explain why each was insufficient. It maps the landscape of AI applications that consume text completions, since these are the services Saturn provisions. It closes with DHCP as an analogy: a protocol that solved the same class of problem—zero-configuration provisioning—at the network layer, and the reasons Saturn operates differently.

2.1 Service Discovery as a Problem

When a device joins a network, it needs to find services. A laptop needs a printer. A phone needs a media server. A coding agent needs a language model. The first two problems were solved decades ago: mDNS lets a printer announce itself, and any device on the same network segment discovers it without configuration. The third problem—discovering AI services—has no standard solution.

Today, connecting to an AI service requires manual work at every point of consumption. A developer building a chat application must obtain an API key from OpenAI or Anthropic, store it in environment variables or configuration files, hardcode the provider’s endpoint URL, and handle billing separately. Syed et al. [40] documented this pattern across the AI-as-a-Service industry: each platform imposes its own account creation, API key management, and billing workflow. An end user who wants access through a self-hosted interface like LibreChat [22] or Open WebUI [29] must repeat this process for each backend. The configuration scales linearly with the number of consumers and providers.

This per-user key management model fails at scale. Meli et al. [23] scanned public GitHub repositories and found over 100,000 containing leaked API secrets. Eighty-one percent of those secrets were never revoked after the authors notified the repository owners, and the best automated scanning tools detected only 25% of leaks. Qazi [34] confirmed that organizations struggle to track their own API keys across teams and services. These findings describe a systemic failure, not individual negligence: static credential management breaks down as the number of credential holders grows.

The question this chapter addresses is whether an existing service discovery protocol could eliminate this manual configuration for AI services, the way mDNS eliminated it for printers.

2.2 Multicast DNS and DNS-Based Service Discovery

Multicast DNS (mDNS), specified in RFC 6762 [8], provides link-local name resolution without infrastructure. Instead of querying a central DNS server, a device sends a query via multicast to address 224.0.0.251 on port 5353 (or `ff02::fb` for IPv6), and any device on the local network segment that owns the queried name responds directly. Names use the `.local` top-level domain—`myserver.local` resolves to an IP address without touching any external DNS infrastructure. Guttman [15] identified this pattern as the foundation of zero-configuration networking: a device joins the network and immediately participates in name resolution.

DNS-based Service Discovery (DNS-SD), specified in RFC 6763 [7], layers structured service advertisement on top of mDNS using standard DNS record types. Three record types compose the discovery flow:

- **PTR records** enumerate available service instances for a given type. A query for `_saturn._tcp.local.` returns pointers to every Saturn service on the network.
- **SRV records** provide the hostname and port for each instance, enabling direct connection.
- **TXT records** carry key-value metadata—arbitrary strings up to 255 bytes each—attached to a service instance.

A client discovers services in three steps: query the PTR record to enumerate instances, read the SRV record for the host and port, and read the TXT record for metadata. This is the same mechanism behind AirPrint, AirPlay, Spotify Connect, and Chromecast: the protocol is mature and deployed on billions of devices.

2.2.1 TXT Records

TXT records are central to Saturn’s design because they carry the metadata that replaces manual configuration. A TXT record contains an ordered set of key-value strings, each limited to 255 bytes. For printers, TXT records advertise capabilities like duplex support or paper sizes. For Saturn, they advertise AI service parameters: the API endpoint URL, the API type (OpenAI-compatible, Ollama, or custom), authentication credentials, priority for load balancing, and optional fields like supported features and rotation intervals. Chapter 3 specifies the full TXT schema.

The 255-byte string limit constrains what metadata Saturn can broadcast. Ephemeral API keys must fit within this limit alongside other fields, which bounds key length to approximately 240 usable characters. This constraint shaped the ephemeral key design: keys are short-lived tokens, not certificates, because the transport imposes a ceiling on payload size.

2.2.2 Platform Implementations

Two implementations dominate mDNS deployment. Apple’s Bonjour [3], released in 2002, ships with every macOS and iOS device and is available as a Windows service. It provides both the system-level mDNS responder and developer APIs for service registration and browsing. Avahi [33], created by Poettering et al. in 2004, provides the equivalent for Linux. Avahi ships as the default mDNS daemon on most Linux distributions, including Ubuntu, Fedora, and Debian, and exposes a D-Bus API that applications use for discovery. Between Bonjour and Avahi, mDNS support is present on every major desktop and server operating system without additional installation.

Siddiqui et al. [36] benchmarked both implementations and found that mDNS service discovery completes within seconds on typical local networks, with Avahi and

Bonjour interoperating correctly across platforms. Siljanovski et al. [37] extended this to resource-constrained devices, demonstrating that mDNS functions on hardware with limited memory and processing power—a result relevant to Saturn’s router deployment on a \$30 MIPS32 device.

2.2.3 Privacy Considerations

mDNS broadcasts are visible to every device on the network segment, which creates a privacy surface. Kaiser and Waldvogel [18] demonstrated that passive eavesdropping on mDNS traffic reveals the services a device offers and consumes, and that this information persists across network sessions. Könings et al. [21] found that 59% of mDNS device names contain the owner’s real name—“John’s MacBook Pro” leaks identity. Kaiser and Waldvogel [19] subsequently proposed a privacy-preserving mDNS extension using pre-shared pairing, though adoption remains limited. For Saturn, these findings mean that service announcements—including the existence of AI endpoints—are visible to any device on the local network. Chapter 6 examines this trade-off in detail.

2.3 Alternative Service Discovery Protocols

Four other protocols address service discovery on local networks. Each was evaluated and rejected for Saturn because none provides the combination of cross-platform support, lightweight metadata, and zero-infrastructure operation that mDNS/DNS-SD offers.

NetBIOS [14] provides name resolution on local networks and remains present in Windows environments for SMB file sharing. It lacks cross-platform support—macOS and Linux have no native implementation—and offers no mechanism for carrying service metadata, only flat name lookups. Microsoft has been deprecating NetBIOS in

favor of mDNS since Windows 10.

WS-Discovery [26] uses multicast SOAP/XML messages to advertise services and was designed for enterprise managed networks. The protocol is verbose: a discovery message requires well-formed XML wrapped in SOAP envelopes, orders of magnitude heavier than a DNS record. Cross-platform library support is sparse compared to mDNS, and no consumer-facing application uses WS-Discovery for service browsing.

UPnP [28] layers a full device control framework—SOAP-based actions, event subscriptions, and XML device descriptions—on top of its own discovery protocol (SSDP). Saturn needs only discovery, not device control, making UPnP’s scope a poor fit. UPnP also carries a documented history of security vulnerabilities, including unauthorized port forwarding and remote code execution, stemming from the complexity of its control surface.

DLNA [11] built media streaming on top of UPnP and inherits its complexity and security profile. The DLNA consortium dissolved in 2017, and the specification is no longer actively developed. Its media-specific scope—designed for audio and video streaming between consumer electronics—does not extend to arbitrary service metadata.

mDNS/DNS-SD avoids each of these limitations. It runs on every major platform through Bonjour and Avahi. Its DNS record format is lightweight. TXT records carry arbitrary key-value metadata. No infrastructure is required. The same protocol already discovers printers, media receivers, and debugging targets across billions of devices.

2.4 AI Applications That Consume Text Completions

Saturn provisions access to AI services that expose the OpenAI-compatible REST API—a de facto standard interface with `/v1/chat/completions`, `/v1/models`, and

/health endpoints. Ollama [27], LiteLLM [5], and OpenRouter [32] all implement this interface, as do dozens of smaller providers. Understanding where these services are consumed clarifies what Saturn replaces.

Self-hosted chat interfaces. LibreChat [22] and Open WebUI [29] provide web-based ChatGPT-like interfaces that connect to user-specified backends. Each requires the user to configure an endpoint URL, select an API type, and supply an API key before any conversation can begin. Jan [16] follows the same pattern as a desktop application. Every instance of every application repeats this configuration independently.

Coding agents. Tools like Cursor [2] and Continue [9] embed AI into development workflows—code completion, refactoring, natural-language queries over codebases. Both allow users to configure custom OpenAI-compatible endpoints, which means each developer on a team must independently obtain and manage API credentials.

Voice transcription. Speech-to-text applications combine an on-device model (typically a Whisper [35] variant) with a cloud language model for transcript cleanup. SuperWhisper [39] runs entirely on-device. Wispr Flow [43] uses a cloud LLM for grammar correction, requiring a subscription. MacWhisper [17] offers both paths. The cloud-dependent variants require their own API configuration, adding a second service endpoint to manage alongside the primary language model.

Development tool discovery. Visual Studio Code already uses mDNS to discover remote debugging targets on the local network [24]—a direct precedent for Saturn’s approach. A developer opens VS Code, and debuggable processes on the LAN appear automatically. Saturn extends this pattern from debugging targets to AI service endpoints.

Across all four categories, the configuration burden is identical: obtain credentials, configure endpoints, manage billing. Saturn replaces this per-application, per-user

work with a single network-level announcement that every application on the network can consume.

2.5 DHCP as Analogy and Contrast

The Dynamic Host Configuration Protocol (DHCP), specified in RFC 2131 [12], is the closest existing analogy to what Saturn does for AI. A device connects to a network, broadcasts a DISCOVER message, receives an OFFER containing an IP address, gateway, and DNS server, and begins operating—all without user configuration. DHCP proved that zero-configuration provisioning works at scale: every network-connected device in the world uses it.

Saturn and DHCP share a philosophy. Both eliminate manual configuration for end users. Both use broadcast or multicast for discovery. Both concentrate complexity in a single administrator who configures a server so that every client on the network benefits automatically. The structural parallel is deliberate: Saturn aims to make “join the network, get AI” as routine as “join the network, get an IP address.”

The protocols differ in four respects. First, DHCP provides configuration once at network join; Saturn services appear and disappear dynamically as beacons start and stop. Second, DHCP assigns one IP address per client; Saturn discovers multiple services with different capabilities, models, and priority levels. Third, DHCP operates at the system level and requires privileged access to configure network interfaces; Saturn operates in userspace, requiring no elevated permissions. Fourth, DHCP carries a fixed set of network parameters (IP, subnet mask, gateway, DNS); Saturn carries extensible metadata about AI service capabilities through TXT records.

These differences explain why DHCP cannot serve Saturn’s purpose directly: it lacks the dynamism, multiplicity, and metadata capacity that AI service discovery

requires. But DHCP validates the design principle: infrastructure-level provisioning eliminates per-user configuration at scale. Saturn applies this principle one layer higher—not to network addresses, but to AI service endpoints.

2.6 The Gap Saturn Addresses

The background surveyed here establishes three facts. First, mDNS/DNS-SD provides cross-platform, zero-infrastructure service discovery with metadata capacity through TXT records. Second, no existing service discovery protocol—NetBIOS, WS-Discovery, UPnP, DLNA, or DHCP—addresses AI service provisioning; each either lacks metadata flexibility, requires infrastructure, targets the wrong scope, or operates at the wrong network layer. Third, every category of AI application that consumes text completions imposes its own configuration burden on every user.

Saturn sits in this gap. It uses mDNS/DNS-SD to broadcast AI service parameters—endpoint URLs, API types, authentication credentials, and capability metadata—so that applications can call `discover()` and connect without any user-supplied configuration. The design and protocol specification follow in Chapter 3.

Chapter 3

Design

Chapter 2 established that mDNS and DNS-SD provide zero-configuration service discovery on local networks, that existing discovery alternatives impose infrastructure or platform dependencies Saturn cannot accept, and that AI service access currently requires per-user credential management. This chapter specifies Saturn’s architecture: the goals it satisfies, the audiences it serves, the concepts it introduces, the protocol it defines, the decisions that shaped it, and the security trade-offs it accepts.

3.1 Design Goals

Saturn’s architecture pursues four goals, each motivated by a deficiency in the current AI provisioning model that Chapter 2 documented.

Zero-configuration consumer access. An end user or application developer who joins a Saturn-enabled network must gain access to AI services without entering URLs, API keys, or account credentials. This goal responds directly to the per-user configuration burden that Syed et al. [40] documented in the AIaaS status quo: accounts, API

keys, and platform lock-in as prerequisites for every consumer. Saturn treats AI access the way DHCP treats IP addressing—joining the network is sufficient.

Administrative centralization. A single administrator configures and maintains AI service availability for all users on the network. The configuration complexity does not disappear; it concentrates in one role rather than replicating across N consumers. This mirrors the DHCP model that Chapter 2 analyzed: one server, zero client configuration.

Backend agnosticism. Saturn must not prescribe what runs behind the API endpoint. A local Ollama instance, a cloud proxy to OpenRouter, and a university-hosted vLLM cluster must all be expressible through the same protocol. This goal ensures Saturn remains useful as the AI inference landscape shifts.

Documented security posture. The architecture must make its security trade-offs explicit and mitigable rather than ignoring them. Broadcasting credentials on a local network is a deliberate choice for the target environment—homes, offices, labs—not an oversight. Section 3.6 specifies the threat models and mitigations.

3.2 Audiences

Saturn defines three distinct roles. Every design decision in this chapter identifies which role bears the complexity and which roles benefit from the simplification.

Administrator. The administrator deploys and configures Saturn beacons. This person selects which AI backends to advertise, sets priorities, manages API credentials for cloud services, and monitors service health. The administrator is the only role that touches configuration files or credentials. In a university department, this is IT staff. In

a home, this is the technically inclined household member. The administrator’s burden increases modestly compared to not running Saturn at all, but that cost is paid once and amortized across every developer and end user on the network.

Application developer. The developer integrates Saturn discovery into software. Rather than hard-coding API endpoints and requiring users to supply credentials, the developer calls a discovery function that returns available services. The developer writes no authentication logic, no configuration UI, and no billing integration. Chapter 4 details the integration patterns across six independent implementations.

End user. The end user interacts with AI-powered applications without awareness of Saturn’s existence. No account creation, no API key entry, no endpoint configuration. The user launches an application, and AI features work because the network provides them. This role performs zero Saturn-specific configuration steps.

3.3 Concepts

Four abstractions structure Saturn’s design. Each is defined here and used consistently throughout the remainder of the thesis.

3.3.1 Endpoints

A Saturn *endpoint* is an HTTP server that exposes an OpenAI-compatible REST API. The server must respond to three paths:

- GET `/health` — returns a JSON object indicating service liveness.
- GET `/v1/models` — enumerates available models in the OpenAI list-models format.

- `POST /v1/chat/completions` — accepts chat completion requests, supporting both streaming and non-streaming responses.

The endpoint need not be OpenAI’s servers. Ollama, LiteLLM [5], vLLM, and OpenRouter [32] all expose this interface. Saturn chose the OpenAI-compatible API because it is the de facto standard: most AI applications already implement a client for it, so Saturn endpoints require zero client-side protocol adaptation. The protocol does not prescribe who operates the server or where it runs—a Raspberry Pi under a desk and a GPU cluster in a data center are equally valid.

3.3.2 Beacons

A *beacon* is a process that announces a Saturn endpoint on the local network via mDNS. Beacons broadcast metadata—the endpoint’s URL, API type, priority, authentication credentials, and supported features—but they do not proxy API traffic. When a client discovers a beacon, it connects directly to the advertised endpoint. The beacon is an announcer, not an intermediary.

This distinction matters for three reasons. First, removing the beacon from the data path eliminates it as a single point of failure: if the beacon process crashes, existing client connections to the endpoint continue functioning. Second, the beacon never observes users’ prompts or model responses, which limits the information exposure surface to the metadata in the mDNS announcement itself. Third, beacon-only architecture means Saturn operates at the discovery layer, not the transport layer, keeping the protocol simple enough for a single developer to implement in any language with an mDNS library.

Beacons support three deployment types. A *local* beacon announces a service running on the same machine or local network, such as an Ollama instance. A *cloud*

beacon announces a remote API service like OpenRouter, distributing ephemeral credentials that authorize network users to access the administrator’s cloud account. A *network* beacon is functionally identical to a local beacon but signals that the service is shared infrastructure rather than a single user’s machine.

3.3.3 Priorities

Each beacon advertises a numeric *priority* value in its TXT record. Lower numbers indicate higher preference. When a client discovers multiple Saturn services, it sorts by priority and selects the lowest-numbered available service.

Priorities give administrators policy control without requiring client-side configuration. An administrator who prefers local inference sets the Ollama beacon’s priority to 1 and the OpenRouter beacon’s priority to 10. Clients automatically prefer the local service. If the local service becomes unavailable, clients fall back to the cloud service without user intervention.

The semantics mirror DNS SRV record priority (RFC 6763 [7]), a convention familiar to network administrators. This is deliberate: Saturn adopts existing conventions rather than inventing new ones wherever the existing convention fits.

3.3.4 Ephemeral Keys

An *ephemeral key* is a time-limited API credential that a cloud beacon generates, broadcasts in its TXT record, and deletes after expiration. Saturn’s default lifecycle uses a ten-minute key lifetime with five-minute rotation intervals, creating an overlap period during which both the current and next key are valid. This overlap ensures zero-downtime credential transitions.

Ephemeral keys address the static API key problem that Meli et al. [23] docu-

mented: over 100,000 GitHub repositories contain leaked secrets, 81% of which are never removed after notification. Static keys fail because every mitigation acts after the damage—detection, rotation, and revocation all lag behind exfiltration. Ephemeral keys sidestep this failure class entirely. A key that expires in ten minutes is worthless by the time it could be extracted from a packet capture, committed to a repository, or shared in a chat log.

The trade-off is operational: ephemeral keys require the beacon to maintain an active session with the cloud provider’s key provisioning API. If the beacon loses connectivity to the provider, it cannot rotate keys, and the current key will eventually expire without replacement. Section 3.6 analyzes this and other failure modes.

3.4 Protocol Specification

Saturn uses DNS-SD over mDNS to advertise AI services. This section specifies the protocol with sufficient precision that an independent implementor can build a conforming client or beacon.

3.4.1 Service Type

Saturn registers services under the DNS-SD service type:

```
_saturn._tcp.local.
```

The underscore prefix follows DNS-SD naming conventions (RFC 6763 [7]). The `_tcp` transport selector indicates the endpoint communicates over TCP (HTTP). The `.local.` domain restricts discovery to the link-local mDNS scope.

3.4.2 DNS Record Structure

Each Saturn service produces three DNS records, following the standard DNS-SD pattern that Chapter 2 described:

1. **PTR record:** Maps the service type `_saturn._tcp.local.` to a named instance (e.g., `Office Ollama._saturn._tcp.local.`). Clients browse for PTR records to enumerate all Saturn services on the network.
2. **SRV record:** Provides the hostname and port of the instance. Clients resolve this to obtain the network address of the endpoint.
3. **TXT record:** Carries key-value metadata describing the service. The TXT record schema is Saturn’s primary contribution at the protocol level.

3.4.3 TXT Record Schema

The TXT record contains key-value pairs encoded per RFC 6763 [7] (each pair as a single DNS TXT string, formatted as `key=value`). Table 3.1 specifies the schema.

The four required fields—`version`, `api_type`, `deployment`, and `priority`—form the invariant core that every conforming beacon must advertise. The `api_base` field is required for cloud deployments, where the endpoint URL cannot be inferred from the local network; for local and network deployments, clients construct the endpoint URL from the SRV record’s hostname and port. Optional fields refine behavior: `ephemeral_key` supplies authentication for cloud backends, `rotation_interval` tells clients when to expect fresh credentials, and `features` enables capability-based filtering.

A constraint inherited from DNS-SD is the 255-byte limit per TXT string (RFC 6763 [7]). Ephemeral keys must fit within approximately 240 usable characters after accounting for the `ephemeral_key=` prefix. This limit has not been a practical constraint—JWT

Table 3.1: Saturn TXT record fields. Fields marked required must appear in every beacon advertisement.

Field	Required	Description
version	Yes	Protocol version (currently 1)
api_type	Yes	Backend API format (e.g., openai)
deployment	Yes	One of local, cloud, or network
priority	Yes	Numeric routing preference; lower is preferred
api_base	Conditional	Base URL of the endpoint. Required when deployment=cloud; for local and network deployments, clients construct the URL from the SRV record’s host and port
ephemeral_key	No	Current API credential; present only for cloud deployments
rotation_interval	No	Key rotation period in seconds (default: 300)
features	No	Comma-separated capability list (e.g., chat,vision,tools)

tokens from OpenRouter and DeepInfra fit within this bound—but it does preclude embedding large payloads such as full X.509 certificates.

3.4.4 Discovery Flow

A Saturn client discovers and connects to a service through four steps:

1. **Browse:** The client sends an mDNS query for PTR records of type `_saturn._tcp.local..` All beacons on the local network respond with their instance names.
2. **Resolve:** For each discovered instance, the client retrieves the SRV record (host-name, port) and TXT record (metadata).
3. **Select:** The client sorts discovered services by priority (ascending), optionally filters by features or deployment type, and selects the preferred service.

4. **Connect:** For cloud deployments, the client uses `api_base` as the endpoint URL and attaches the `ephemeral_key` as a Bearer token. For local and network deployments, the client constructs the endpoint URL from the SRV record's host-name and port (e.g., `http://{host}:{port}/v1`) and calls the endpoint's API directly.

The entire flow executes without user interaction. From the application's perspective, discovery replaces what would otherwise be a configuration file or environment variable containing an API URL and key.

3.5 Architecture Decisions

Three decisions shaped Saturn's design. Each is presented with its rationale and the alternative it superseded.

3.5.1 Beacon, Not Proxy

Saturn beacons announce endpoints but do not relay API traffic. The alternative—a proxy architecture where all requests flow through a central server—would provide richer traffic control (rate limiting, logging, load balancing) but would introduce a single point of failure, require the proxy to handle streaming responses and large payloads, and place the proxy operator in a position to observe every prompt and response.

I chose the beacon architecture because Saturn's primary goal is zero-configuration access, not traffic management. A beacon requires only an mDNS library and a configuration file. A proxy requires an HTTP server, TLS termination, connection pooling, and streaming support—infrastructure complexity that contradicts the goal of enabling a single administrator to deploy Saturn on commodity hardware. The GL.iNet travel

router described in Chapter 4 runs Saturn’s beacon alongside DHCP and DNS on a MIPS32 processor with 128 MB of RAM. A proxy architecture would not fit within those constraints.

The trade-off is that Saturn cannot enforce per-user rate limits, audit usage, or perform request-level access control. These are features the beacon architecture deliberately omits. Deployments that require them can place a rate-limiting reverse proxy in front of the endpoint itself, outside Saturn’s scope.

3.5.2 OpenAI API as Interface Standard

Saturn requires endpoints to expose the OpenAI-compatible REST API rather than defining its own interface format. The alternative—a Saturn-specific API schema—would have allowed tighter integration and custom metadata in responses, but would have required every backend and every client to implement a new protocol.

The OpenAI API won because it already exists and is already implemented. Ollama [27], LiteLLM [5], OpenRouter [32], and vLLM all expose it. Applications like Open WebUI [29], Jan [16], and Cursor [2] already consume it. By adopting the existing standard, Saturn makes any OpenAI-compatible backend immediately usable as a Saturn endpoint and any application that supports “custom OpenAI endpoint” immediately usable as a Saturn client—in both cases with zero code changes to the backend or application.

The trade-off is dependence on a format defined by a single company. If OpenAI changes the API in incompatible ways, Saturn inherits the disruption. In practice, the `/v1/chat/completions` interface has remained stable since its introduction, and the ecosystem’s investment in compatibility (dozens of independent reimplementations) creates strong inertia against breaking changes.

3.5.3 Priority as Policy

Saturn uses a single numeric priority field for service selection rather than a multi-attribute ranking system. The alternative—weighted scoring across latency, cost, model capability, and availability—would enable more sophisticated routing but would require clients to implement a scoring engine and administrators to configure weights.

A single priority field suffices because Saturn targets environments where the administrator knows the preferred routing order. In a lab with a local GPU server and a cloud fallback, the administrator sets the local service to priority 1 and the cloud service to priority 10. The routing policy is expressed in two integers rather than a scoring matrix. Clients implement selection as a sort-and-filter operation, which any language can express in a few lines of code.

The trade-off is inflexibility: priority cannot express “prefer the local service unless latency exceeds 500 ms” or “use the cheapest service that supports vision.” These policies require client-side logic that Saturn’s minimal protocol does not provide. I consider this acceptable because Saturn’s contribution is discovery, not orchestration. An application that needs sophisticated routing can use Saturn to discover available services and then apply its own selection logic to the results.

3.6 Security

Saturn deliberately trades enterprise-grade verification for zero-configuration access. This section specifies the two threat models the architecture addresses and the mitigations it provides, rather than deferring security analysis to later chapters. An engineer evaluating whether to deploy Saturn on a given network needs this information alongside the protocol specification.

3.6.1 Threat Model 1: Corporate Data Collection

The first threat is that a user’s AI interactions—prompts, completions, conversation histories—flow to a cloud provider who retains, analyzes, or monetizes them. This threat motivates users who distrust companies like OpenAI or Anthropic with their data but still want access to capable models.

Saturn addresses this threat through backend agnosticism. Because the protocol does not prescribe what runs behind the endpoint, an administrator can deploy a local inference server (Ollama with an open-weights model) and set its priority higher than any cloud beacon. Users on the network automatically route to the local service. Their prompts never leave the LAN.

The limitation is capacity. Local inference requires hardware that matches or approximates the capability of cloud models. A university department can justify a GPU server; a household typically cannot. When the local model is insufficient and the user falls back to a cloud endpoint, Saturn provides no additional data protection beyond what the cloud provider offers. Saturn enables the administrator to make local inference the default path, but it cannot guarantee that path is always available.

3.6.2 Threat Model 2: Untrusted Administrator

The second threat is that the network administrator—the person who deploys Saturn beacons—is untrustworthy. In an office, the sysadmin controls the beacon and could configure it to route traffic through a logging proxy, advertise a malicious endpoint, or retain copies of ephemeral keys.

Saturn provides limited mitigation for this threat because the administrator is, by design, the trusted party. The beacon architecture reduces the administrator’s observational power compared to a proxy: because API traffic flows directly from client to

endpoint, the administrator who runs only a beacon cannot see prompts or responses. The administrator sees only the metadata broadcast in TXT records, which the administrator created in the first place.

An administrator who controls both the beacon and the endpoint, however, has full access to all traffic. Saturn cannot protect against this scenario without introducing user-level authentication, which would destroy the zero-configuration property. This is the fundamental trade-off the architecture makes: Saturn trusts the network in the same way that connecting to office WiFi trusts the network operator with DNS resolution and IP routing. Ward and Beyer [41] argued in *BeyondCorp* that networks should never be trusted; Saturn disagrees for its target environment of homes, offices, and campus labs where the network operator is a known party and the alternative—per-user credential management—creates the access barriers Saturn exists to remove.

3.6.3 Broadcast Exposure

Any device on the local network can observe Saturn’s mDNS announcements. Kaiser and Waldvogel [18] demonstrated that mDNS-SD traffic reveals service metadata to all network participants, and Könings et al. [21] found that 59% of mDNS device names contain real user names. Saturn inherits these exposure properties.

For local deployments (no `ephemeral_key` in the TXT record), the exposed information is limited to the existence and location of AI services on the network—comparable to what a network scan would reveal. For cloud deployments, the TXT record also contains an ephemeral API key. Any device on the network can read this key and use it to make API calls against the administrator’s cloud account.

Saturn mitigates financial exposure through two mechanisms. First, ephemeral keys expire in ten minutes. An attacker who captures a key has a narrow window to exploit

it, and the key self-destructs whether or not the attacker is detected. Second, administrators can set spending limits on their cloud provider accounts, capping the maximum damage from unauthorized key use regardless of how many keys are captured.

These mitigations do not eliminate the threat; they bound its impact. A deployment that requires stronger guarantees—preventing unauthorized users from making any API calls—must add network-level access control (e.g., VLAN segmentation or 802.1X authentication) outside Saturn’s scope. Kaiser and Waldvogel [19] proposed privacy-preserving extensions to mDNS-SD that could enable encrypted service discovery without breaking zero-configuration semantics, a direction Chapter 6 explores as future work.

3.7 Summary

Saturn’s design addresses the problem of AI service provisioning through four architectural choices: beacon-based announcement rather than traffic proxying, the OpenAI-compatible API as a shared interface standard, numeric priority for administrator-controlled routing, and ephemeral keys for time-limited credential distribution. The protocol registers services under `_saturn._tcp.local.`, carries all connection parameters in DNS-SD TXT records, and enables a four-step discovery flow that executes without user interaction. The security model trades network-level trust for zero-configuration access, a trade-off appropriate for Saturn’s target environments of homes, offices, and campus labs. Chapter 4 describes how six independent implementations realize this design across multiple languages and platforms.

Chapter 4

Implementation

Saturn’s design, described in Chapter 3, specifies a protocol (Section 3.4) and an architecture (Section 3.5). This chapter demonstrates that the protocol works by presenting three implementations that progressively expand Saturn’s reach: a VLC media player extension that brings AI to a non-AI-native desktop application, a router firmware integration that provisions AI at the network infrastructure layer, and a fork of the OpenCode coding agent that handles the demands of agentic tool calling and multi-turn conversation. Each implementation targets a different class of application, and together they validate argument **A1**: six cross-platform implementations prove that mDNS/DNS-SD AI provisioning works.

Before describing these three integrations, I first summarize the shared foundation they build on — the core Python package and the TypeScript AI SDK provider — because the reader needs to understand what was reused before they can evaluate what was built.

4.1 Shared Foundation

Two packages provide the discovery and API translation layers that all three integrations consume.

The `saturn-ai` Python package (published on PyPI, v1.0.1) implements the server side of the Saturn protocol. It advertises services on `_saturn._tcp.local.` via the `zeroconf` library, manages ephemeral credential lifecycles through provider-specific modules for OpenRouter and DeepInfra, and exposes OpenAI-compatible HTTP endpoints (`/v1/health`, `/v1/models`, `/v1/chat/completions`) regardless of the actual backend. An administrator runs a single command — `saturn run openrouter` — and the package handles TOML configuration loading, mDNS registration, credential rotation, and HTTP proxying. On the consumer side, `discover()` listens for mDNS responses, parses TXT records, and returns a priority-sorted list of `SaturnService` objects. No URLs, API keys, or configuration files are required from the consumer.

The `ai-sdk-provider-saturn` npm package (v0.1.4) implements the client side for the TypeScript ecosystem. It conforms to Vercel’s AI SDK ProviderV3 interface, resolving endpoint configuration, authentication, and model routing entirely at runtime from mDNS broadcasts. The package uses the `multicast-dns` npm library — a pure JavaScript mDNS implementation with no native dependencies — to query `_saturn._tcp.local.`, parse TXT records, and maintain a live service registry. A developer writes `import { saturn } from 'ai-sdk-provider-saturn'` and calls `generateText({ model: saturn('gpt-4o'), prompt: '...' })`. No API key, endpoint URL, or environment variable appears in the code.

These two packages use different mDNS libraries in different languages — Python’s `zeroconf` and JavaScript’s `multicast-dns` — yet interoperate on the same `_saturn._tcp.local.` service type and TXT record schema. This cross-language interoperability, achieved

without any bridging layer, supports argument **A2**.

4.2 VLC Extension: AI in a Non-AI-Native Application

Saturn claims that zero-configuration AI provisioning is feasible (**V1**). A protocol that works only inside developer tools and CLI scripts would be a weak proof of that claim. VLC media player — one of the most widely installed desktop applications — has never offered AI features. If Saturn can bring AI capabilities into VLC without the user configuring an API key, endpoint URL, or model name, the protocol demonstrates practical value beyond its original development context.

4.2.1 Architecture

VLC’s extension system accepts only Lua scripts with limited HTTP capabilities: `vlc.stream()` supports GET requests but not POST. VLC provides no native mDNS support. These constraints rule out embedding Saturn discovery directly in the extension.

I solved this with a two-layer architecture. The outer layer is a Lua extension (`saturn_chat.lua`, v1.5.1) that provides an in-player GUI dialog — service and model dropdowns, a chat input, and styled HTML output. The inner layer is a Python/FastAPI bridge (`vlc_discovery_bridge.py`) that performs mDNS service discovery, health-monitors discovered services, aggregates their models, and exposes an OpenAI-compatible REST API on localhost. The Lua extension launches the bridge as a background process on activation, communicates with it over HTTP, and shuts it down on deactivation.

This bridge pattern addresses a constraint that extends beyond VLC. Most desktop applications will never add native mDNS discovery. The bridge demonstrates that Saturn can integrate with any application capable of making HTTP requests to the endpoints specified in Section 3.3.1, regardless of its extension language or networking

capabilities.

4.2.2 Technology Choices

The bridge uses FastAPI on uvicorn because the Lua extension needs a local HTTP API, and FastAPI provides OpenAI-compatible endpoint routing with minimal code. For mDNS discovery, the bridge invokes macOS's `dns-sd` command-line tool via subprocess rather than importing the `zeroconf` Python library. This avoids a `zeroconf` dependency that would complicate PyInstaller bundling. The `dns-sd` tool ships with every macOS installation and performs the same `_saturn._tcp.local.` browse-and-lookup sequence as the core package. The bridge is packaged into a standalone executable via PyInstaller so that end users need no Python installation.

Because VLC's Lua environment cannot issue POST requests, the extension sends chat payloads as URL-encoded JSON in GET query parameters. The extension also includes a hand-written recursive descent JSON parser and encoder, since VLC Lua provides no JSON library.

4.2.3 Walkthrough: From Launch to Chat

When a user activates the extension through VLC's menu (View → Extensions → Saturn Chat), the following sequence occurs:

1. The extension detects the host operating system and locates the bundled bridge executable in its installation directory.
2. It launches the bridge in the background, passing a `--port-file` argument that specifies where the bridge should write its address.

3. The bridge auto-detects an available port starting from 9876, starts the HTTP server, and writes `host:port` to the port file.
4. The extension polls the port file for up to ten seconds, reads the bridge address, and begins health-checking the bridge with exponential backoff (seven retries over approximately seven seconds).
5. Once the bridge reports “ready,” the extension queries `/services` and `/v1/models` to populate its service and model dropdowns.
6. The bridge, meanwhile, has launched a background discovery thread that browses `..saturn..tcp.local.` every ten seconds and health-monitors each discovered service.
7. The user selects a service and model, types a message, and the extension sends it to the bridge’s `/v1/chat/completions` endpoint. The bridge proxies the request to the selected Saturn service and returns the response.
8. Media context — title, artist, album, genre, and current playback position — is extracted automatically from VLC’s input metadata via `vlc.input.item():metas()` and included in the system prompt, so the AI knows what the user is watching without being told.

This walkthrough demonstrates argument **A3**: the consumer path is zero-configuration. The user copies a directory, clicks a menu item, and chats with AI about their media. No account creation, no API key, no endpoint URL.

4.2.4 What the VLC Extension Proves

The VLC extension establishes that Saturn works in applications that are not designed around AI. VLC is a media player. Its extension system was built for playlist manip-

ulation and media analysis, not for language model integration. The bridge pattern overcomes VLC’s architectural constraints — Lua-only extensions, GET-only HTTP, no JSON library, no mDNS support — without modifying VLC itself. This matters because the majority of applications that could benefit from AI features share similar constraints: limited extension environments with no native service discovery.

4.3 Saturn Router: AI at the Network Infrastructure Layer

The VLC extension proved that Saturn can bring AI to a consumer application. The next question was whether Saturn could operate at the network level — embedded in the device that defines the network itself. If Saturn runs on a router alongside DHCP and DNS, then every device that joins the network gains access to AI services, just as every device gains an IP address and name resolution.

4.3.1 Architecture

The Saturn router implementation consists of three layers: a Rust binary that manages the Saturn protocol, an OpenWRT integration layer that embeds that binary into the router’s service management framework, and a LuCI web interface that gives administrators a browser-based configuration panel.

The Rust binary (`saturn crate, v0.1.0`) contains four source files. `config.rs` deserializes JSON configuration with hand-rolled CLI argument parsing — no `clap` dependency, keeping binary size minimal. `mdns.rs` wraps the `mdns-sd` crate’s `ServiceDaemon` to register and unregister `_saturn._tcp.local.` services. `providers/provider.rs` manages health checks against upstream AI endpoints, generates ephemeral credentials

via configurable HTTP templates, rotates those credentials on schedule, and cleans up expired keys through the provider's API. `main.rs` ties these together in an event loop that ticks every 100 milliseconds, rotating credentials or checking health depending on the operating mode, and registering or unregistering the mDNS service based on backend availability.

4.3.2 Technology Choices

I chose Rust because the target hardware — a GL.iNet GL-MT300N-V2 travel router with a MIPS32 processor and approximately 128 MB of RAM — cannot run a Python interpreter. The release profile aggressively optimizes for size: `opt-level = "z"`, link-time optimization, single codegen unit, `panic = "abort"`, and symbol stripping. TLS support uses `rustls` rather than `native-tls` because cross-compilation to `mipsel-unknown-linux` (a Rust Tier 3 target) requires a musl-compatible TLS stack. The MIPS32 processor lacks a floating-point unit, so the cross-compilation toolchain uses a soft-float musl variant (`mipsel-linux-muslsf`).

The cross-compilation environment is a Docker container that installs `gcc-mipsel-linux-gnu` and the soft-float musl toolchain, then invokes Rust's nightly `-Zbuild-std` to compile the standard library from source for the target. A GitHub Actions workflow builds the binary on tag pushes and publishes it as a release artifact with SHA256 checksums.

For HTTP requests, the binary uses `attohttpc` rather than `reqwest` because `attohttpc` has a smaller dependency tree suitable for embedded targets. The `mdns-sd` crate — a third independent mDNS library, distinct from Python's `zeroconf` and JavaScript's `multicast-dns` — handles service announcement and deregistration.

4.3.3 OpenWRT Integration

The OpenWRT integration embeds Saturn into the router’s service management infrastructure through three components.

A `procd` init script manages the binary’s lifecycle. It supports multiple simultaneous services: each UCI `config service` section spawns a separate process with its own JSON configuration file (written to `/tmp/saturn.d/` with `chmod 600` permissions). The init script auto-assigns ports for cloud services to avoid conflicts, checks for existing port usage via `netstat`, and implements a respawn policy (five failures within one hour before giving up). If the binary is missing, `ensure_binary()` detects the router’s architecture and downloads the correct build from GitHub Releases — the administrator does not need to transfer the binary manually.

A UCI configuration schema defines the administrator’s interface. Each service section specifies a name, deployment type (cloud or network), API type (OpenAI or Ollama), base URL, API key, and optional ephemeral key parameters. This is the same `uci set/uci commit` workflow that OpenWRT administrators use for firewall rules, wireless settings, and DHCP configuration.

A LuCI web application provides a browser-based GUI. The JavaScript form (`services.js`) handles field validation (service name format, URL format, port ranges, expiration exceeding rotation interval), dynamic field visibility (cloud-specific fields hide for network deployments), and live status badges that poll every ten seconds. The RPC backend is a shell script that checks process health via `curl` with five-second timeouts, tests endpoint connectivity, and reports model counts. Saturn appears under LuCI’s “Services” menu category — the same category as DHCP, DNS, and firewall rules.

4.3.4 Walkthrough: Deploying Saturn on a Router

An administrator configures Saturn through the LuCI web interface:

1. The administrator navigates to Services → Saturn in the LuCI menu and clicks “Add Service.”
2. They fill in four fields: a service name, the deployment type (cloud), the upstream API base URL, and the API key environment variable. For ephemeral key support, they additionally set rotation and expiration intervals.
3. They click “Save & Apply.” The init script generates a per-service JSON configuration, downloads the Saturn binary if needed, and starts a procd-managed process.
4. The Rust binary performs an initial health check against the upstream API. If the service is healthy and ephemeral keys are enabled, it generates the first credential by POSTing to the provider’s key endpoint with a templated request body.
5. The binary registers the service on `_saturn._tcp.local.` via the `mdns-sd` crate, embedding the endpoint URL, API type, priority, and (if applicable) the ephemeral key in the mDNS TXT records (the schema defined in Table 3.1).
6. Every device on the network now receives the announcement through standard multicast DNS — the same mechanism that advertises printers and Chromecast devices.
7. The status badge in LuCI turns green. The administrator sees “UP” next to the service name.

If the upstream API becomes unavailable, the binary’s health check detects the failure and unregisters the service from mDNS. Clients stop discovering it. When health recovers, the binary re-registers. This health-based registration ensures that clients never discover an unhealthy backend.

In ephemeral key mode, the main loop rotates credentials on schedule (default: every five minutes), re-registers the mDNS service with updated TXT records containing the new key, and deletes the previous key through the provider’s API. The Drop implementation on `SaturnProvider` ensures that ephemeral keys are deleted even on unexpected termination — a defense-in-depth measure against credential leakage. This lifecycle implements the ephemeral key design described in Section 3.3.4 and supports argument **A5**: ephemeral keys with ten-minute expiry and five-minute rotation sidestep the static key leakage class that Meli et al. [23] documented.

4.3.5 What the Router Proves

The router implementation transforms Saturn’s “AI as network infrastructure” framing from analogy to architecture. Saturn runs on the same device as DHCP and DNS, managed by the same service framework (`procd`), configured through the same admin interface (LuCI), and listed in the same menu category (Services). The binary runs on a \$30 MIPS32 travel router with no floating-point unit — hardware that defines the lower bound of what consumer networking equipment can do. This supports argument **A7**: Saturn runs on constrained embedded hardware alongside existing network services.

The router also provides the strongest evidence for the administrator experience described in argument **A4**. One administrator fills in a LuCI form. Every device on the network — laptops, phones, tablets, VLC extensions, coding agents — discovers the service automatically. The configuration complexity is centralized in one place rather

than distributed across every user and every application.

4.4 OpenCode: Saturn in an AI Coding Agent

The VLC extension proved Saturn works in non-AI applications. The router proved it works at the network infrastructure layer. The final question was whether Saturn could handle the demands of a complex AI-native application — one that requires streaming responses, structured tool calling, multi-turn conversation, and file editing.

OpenCode is an open-source AI coding agent built by Anomaly. It provides a terminal UI and web interface for AI-assisted software development, with features including LSP integration, provider-agnostic model routing, session persistence, and agentic tool use. I forked OpenCode and integrated `ai-sdk-provider-saturn` as a bundled provider. The result, OpenCode-Saturn, discovers Saturn services on the local network via mDNS and makes their models available for code generation, editing, and tool calling — with no API keys, endpoint URLs, or environment variable configuration.

Some readers may object that integrating Saturn into an AI-native application is circular: of course an AI tool can use an AI discovery protocol. The VLC extension answers that objection. VLC is a media player with no AI heritage. Saturn brought AI to VLC through the bridge pattern. OpenCode-Saturn demonstrates the complementary point: Saturn handles the full complexity of agentic AI workflows, not just simple chat.

4.4.1 Integration Approach

Rather than forking OpenCode’s core abstractions, I extended its existing provider system. OpenCode maintains a `CUSTOM_LOADERS` record that maps provider names to initialization functions. I added a `saturn` entry that creates a `createSaturn()` SDK instance with a three-second discovery timeout, waits for mDNS responses, fetches

/v1/models from all discovered services, and registers each service as a dynamic provider with its models. The integration required changes across twelve files in four packages, but none of those changes modified OpenCode’s core model routing, session management, or UI rendering.

4.4.2 Dynamic Registration

Static provider systems assume that the set of available models is fixed at startup. Saturn’s mDNS-based discovery means models can appear and disappear at runtime — an administrator might start a new beacon, or a router might go offline. OpenCode’s architecture was not designed for this.

I implemented dynamic registration through two callback functions. When the `ai-sdk-provider-saturn` library detects a new service on the network, `onServiceDiscovered` fetches its models (with up to three retries and 500-millisecond delays), creates a provider entry, and calls `registerDynamic()` to add it to OpenCode’s state. When a service disappears, `onServiceRemoved` verifies the removal (guarding against race conditions where a service is simultaneously re-discovered), then calls `unregisterDynamic()` to remove it.

Both functions emit events on two buses: an instance-scoped `Bus` for the current project session and a process-wide `GlobalBus` for cross-instance updates. These events propagate to both the terminal UI and web UI, updating model selectors in real time.

4.4.3 Integration Challenges

Integrating Saturn into OpenCode surfaced six bugs, each revealing an assumption in the host application that Saturn’s runtime discovery model violates.

The most instructive was the `finishReason` format mismatch. OpenCode’s ses-

sion processor expected the Vercel AI SDK v2 format — a plain string like "stop" or "length". Saturn's provider returns the v3 format: an object { unified: "stop", raw: "end_turn" }. After the first response, subsequent messages queued instead of sending because the processor could not parse the finish signal. The fix was a single line of type normalization, but finding it required tracing a silent failure through the session pipeline.

A second critical bug involved tool calling. Saturn-discovered models initially lacked the `toolCalling` capability flag. Without it, OpenCode serialized structured function calls as plain text rather than invoking the tool-calling protocol. The agent could chat but could not edit files or run commands — the core value proposition of a coding agent. Setting `toolcall: true` in the dynamic model registration restored full agentic functionality.

A third issue arose from Bun's default HTTP timeout. Long-running LLM streaming requests — common in coding agents that generate large code blocks — were killed before completion. I worked around this by monkey-patching `globalThis.fetch` for the duration of each streaming call, setting `timeout: false`, and restoring the original `fetch` in a `finally` block.

These bugs are evidence, not embarrassments. They demonstrate that Saturn's protocol works end-to-end — the failures occurred in the host application's assumptions about provider lifecycle, not in the discovery or API translation layers.

4.4.4 What OpenCode Proves

OpenCode-Saturn validates that Saturn handles the most demanding AI integration pattern: agentic tool calling with streaming responses, multi-turn conversation, dynamic model discovery, and cross-service failover. When an administrator deploys a new

Saturn beacon advertising a model, OpenCode-Saturn picks it up live without restart. When a service fails, the failover function searches all other `saturn:*` providers for the same model and reroutes transparently.

The integration also demonstrates Saturn’s composability. OpenCode-Saturn depends on `ai-sdk-provider-saturn`, which depends on `multicast-dns`. The protocol’s SDK layer absorbed the complexity of mDNS discovery, TXT record parsing, ephemeral key refresh, and circuit breaking. The application-level integration dealt only with provider registration, event propagation, and capability flags — concerns specific to OpenCode’s architecture, not to Saturn’s protocol.

4.5 Summary of Implementations

Table 4.1 summarizes the three implementations and their relationship to Saturn’s thesis claims.

Implementation	Language	mDNS Library	Li-	What It Demonstrates
VLC Extension	Lua / Python	macOS dns-sd CLI		AI in a non-AI-native consumer application via bridge pattern
Saturn Router	Rust	mdns-sd crate		AI provisioning at the network infrastructure layer on constrained hardware
OpenCode Fork	TypeScript	multicast-dns npm		Full agentic AI workflow with tool calling, streaming, and dynamic discovery

Table 4.1: Three Saturn implementations demonstrating progressive scope expansion. Each uses a different mDNS library, supporting argument **A10**: seven consumers across three languages and four mDNS libraries interoperate on `_saturn._tcp.local.` with no bridging layer.

Together with the core Python package, the TypeScript AI SDK provider, the Open WebUI plugin, and the MCP server, these three implementations bring Saturn’s total to seven consumer integrations across three languages. The progression from VLC to

router to OpenCode was deliberate: each implementation answered a question left open by the previous one, and together they establish that Saturn's protocol is general enough to serve applications ranging from media players to network infrastructure to AI coding agents.

Chapter 5

Evaluation

The Introduction advanced three claims about Saturn: that zero-configuration protocols can provision AI services (**V1**), that network-level provisioning reduces total configuration effort (**V2**), and that the security trade-offs of broadcast discovery are documented and addressable (**V3**). This chapter presents the evidence for each claim. I evaluate **V1** through an implementation census and protocol analysis, **V2** through a cognitive walk-through following the methodology of Wharton et al. [42], and **V3** through a threat comparison grounded in the mDNS security literature. A heuristic evaluation using Nielsen’s ten usability heuristics [25] then cuts across all three claims to assess Saturn’s usability surface. The chapter closes with threats to validity.

All three evaluations are analytical rather than empirical. Saturn has no user base, no production deployment, and no institutional partner willing to run a field study. Analytical methods — protocol analysis, cognitive walkthroughs, threat modeling — are appropriate for a system at this stage because they evaluate the design and its properties without requiring participants. Where analytical methods limit what I can conclude, I state those limits explicitly.

5.1 Claim 1: Zero-Configuration AI Provisioning Is Feasible

V1 asserts that mDNS/DNS-SD can provision AI services without end-user configuration. I decompose this claim into three testable requirements and evaluate each through implementation evidence.

5.1.1 Methodology

Feasibility claims require existence proofs: at least one working system that satisfies the stated requirements. I decompose **V1** into three requirements:

R1 — Service Discovery. Saturn services can be found with zero configuration on the consumer side.

R2 — Connection Sufficiency. The information carried by mDNS discovery provides everything needed for a working API call.

R3 — Cross-Implementation Interoperability. Independent implementations, written in different languages using different mDNS libraries, consume the same protocol without a bridging layer.

Each requirement maps to specific arguments from the argument registry: **R1** to **A1**, **A3**, and **A7**; **R2** to **A8** and **A9**; **R3** to **A10** and **A2**.

5.1.2 Results

R1: Service Discovery. Six independent implementations discover Saturn services through standard mDNS/DNS-SD queries on the `_saturn._tcp.local.` service type.

The core Python package calls `discover()`, which listens for mDNS responses and returns `SaturnService` objects containing endpoint URLs, API types, and priorities. The Rust router (Section 4.3) runs on a \$30 GL.iNet MIPS32 device alongside DHCP and DNS, advertising Saturn services as part of the network infrastructure itself (A7). The VLC extension (Section 4.2), the Open WebUI plugin, the MCP server, and the Vercel AI SDK provider each discover services through their respective mDNS libraries without any user-supplied URLs, API keys, or configuration files (A3).

R2: Connection Sufficiency. Saturn’s TXT record schema (Section 3.4.3) carries all parameters required to establish an API connection: `api_base` for the endpoint URL, `api_type` for the backend protocol, `priority` for service selection, and `deployment` for the hosting context (A8). Three distinct API backends — Ollama (no authentication), DeepInfra (static JWT), and OpenRouter (ephemeral key with rotation) — operate through this same schema without out-of-band configuration (A9). A consumer that calls `discover()` and selects a service by priority receives a complete, working connection specification.

R3: Cross-Implementation Interoperability. Seven consumers spanning three programming languages and four independent mDNS libraries interoperate on `_saturn._tcp.local.` (A10). The Python implementations use the `zeroconf` library. The Rust router uses `mdns-sd`. The TypeScript AI SDK provider uses `multicast-dns`. The VLC extension invokes the macOS `dns-sd` CLI. No shared Saturn-specific code bridges these libraries (A2); each parses standard DNS-SD TXT records independently. The protocol, not a shared library, is the interoperability layer.

5.1.3 Assessment

Saturn satisfies all three requirements. R1 holds across six implementations on three platforms. R2 holds across three API backends with different authentication models. R3 holds across four mDNS libraries with no bridging code. Together, these results establish that mDNS/DNS-SD can carry AI service metadata sufficient for zero-configuration provisioning.

One qualification applies. The TXT record schema drifts across implementations: the `auth` field exists only in the TypeScript provider, and the Rust router omits `capabilities`, `cost`, and `context` fields that the Python core defines. The four core fields (`version`, `deployment`, `api_type`, `priority`) are consistent across all implementations, and these four suffice for discovery and connection. The optional field drift does not break interoperability, but it does constrain the metadata richness available to consumers that depend on non-core fields.

5.2 Claim 2: Reduced Configuration Effort

V2 asserts that Saturn reduces total configuration effort compared to per-user manual setup. The claim is comparative and honest about a trade-off: the administrator does more work so that N developers and M end users do less.

5.2.1 Methodology

I evaluate **V2** using a cognitive walkthrough, an analytical inspection method introduced by Wharton et al. [42]. The walkthrough traces task completion step by step across defined personas, counting discrete configuration actions required to reach a completion criterion. The completion criterion for both conditions is: *an end user on*

the network uses an AI-powered feature in an application.

I define three personas that reflect Saturn’s role model:

- **Sysadmin** — provisions the AI service and makes it available on the network.
- **App Developer** — integrates AI capabilities into an application.
- **End User** — consumes AI features through the developer’s application.

Two conditions are compared. In the *traditional* condition, the sysadmin obtains an API key from a cloud provider and distributes it to developers, who configure their applications with the key and expose AI features to end users, who create accounts and provide payment credentials. In the *Saturn* condition, the sysadmin runs a Saturn beacon that advertises the service via mDNS; developers call `discover()` instead of configuring keys and URLs; end users do nothing.

The walkthrough scripts are implemented as annotated Python files in the project repository (`cog-walkthrough/`), where each configuration action corresponds to a documented step. This makes the step counts reproducible: an independent reviewer following the same scripts arrives at the same counts.

I chose a cognitive walkthrough over a user study for two reasons. First, Saturn has no deployed user base from which to recruit participants. Second, the relevant comparison is structural — how many discrete actions each role must perform — rather than behavioral. A walkthrough captures the configuration topology; a user study would capture timing and error rates that require participants.

5.2.2 Results

Table 5.1 reports step counts per persona under each condition (**A11**).

Table 5.1: Cognitive Walkthrough Step Counts by Persona

Persona	Traditional	Saturn	Change
Sysadmin	12	14	+2 (+17%)
App Developer	19	4	−15 (−79%)
End User	7	0	−7 (−100%)
Total (1+1+1)	38	18	−20 (−53%)

The sysadmin performs two additional steps under Saturn (+17%) because Saturn requires running a beacon process in addition to obtaining the API key (**CA3**). The app developer performs fifteen fewer steps (−79%) because Saturn eliminates the entire billing integration, API key management, and endpoint configuration stack (**A12**). The end user performs zero steps (−100%) because Saturn removes the account creation and payment credential entry that the traditional model requires (**A13**).

With one persona of each role, total steps drop from 38 to 18, a 53% reduction. The reduction grows with scale. For N app developers and M end users, the formulas are (**A14**):

$$\text{Traditional} = 12 + 19N + 7M$$

$$\text{Saturn} = 14 + 4N + 0M$$

At $N = 10$ developers and $M = 100$ end users, the traditional model requires 902 steps; Saturn requires 54 — a 94% reduction. The Saturn formula has no M term because end users perform zero configuration steps regardless of how many there are.

5.2.3 What Drives the Reduction

The 79% developer reduction stems from a specific structural change: Saturn eliminates the billing and payment integration stack. In the traditional condition, the app developer

must create a provider account, generate an API key, store the key securely, configure the application with the key and endpoint URL, implement billing logic or pass-through payment, and handle key rotation. Under Saturn, the developer calls `discover()`, selects a service by priority, and makes API calls against the returned endpoint. The entire key-management and billing layer vanishes because the sysadmin’s beacon already carries the credentials.

The 100% end-user reduction follows from the same structural shift. Traditional AI access requires end users to create accounts and provide payment information. Saturn services require neither: the network provides the AI capability, and the application connects to it transparently.

5.2.4 Assessment

Saturn reduces total configuration effort across the three-persona model by 53% at minimum scale and 94% at moderate scale. The reduction is not free — it shifts two additional steps to the sysadmin. This trade-off is the core of **V2**: complexity centralizes in one administrator rather than scattering across N developers and M users.

5.3 Claim 3: Security Trade-offs Are Known and Addressable

V3 asserts that the security and privacy trade-offs of broadcast-based AI discovery are documented in existing literature and can be mitigated without destroying the zero-configuration property. This claim is deliberately conservative: I do not argue that Saturn is secure in an enterprise sense, but that its threat surface is understood and that mitigations exist.

5.3.1 Methodology

I evaluate **V3** through three analytical lenses: an information leakage analysis of Saturn’s protocol surface, a STRIDE threat comparison between Saturn’s ephemeral credential model and the static API key model it displaces, and an exposure window analysis grounded in the secret leakage data of Meli et al. [23].

The analysis is protocol-level, derived from the TXT record schema defined in Section 3.4.3 and the credential lifecycle described in Section 3.3.4 and Chapter 4. I did not conduct empirical penetration testing or packet capture experiments. Protocol analysis is appropriate because the security properties under evaluation — what the protocol exposes by design and how credential lifetimes compare — are deterministic properties of the specification, not emergent behaviors requiring runtime observation.

5.3.2 Results

Information Leakage Surface. Saturn’s mDNS TXT records (Section 3.4.3) expose eleven metadata fields by protocol design: `version`, `deployment`, `api_type`, `priority`, `api_base`, `capabilities`, `cost`, `context`, `model`, `auth`, and `ephemeral_key`. In the base (local) configuration — where the backend runs on the same network as the clients — zero of these fields contain credential material. The `api_base` field reveals the endpoint URL, and `api_type` reveals the backend protocol, but neither constitutes a secret.

Cloud deployments add one credential field: `ephemeral_key`, a time-limited API key with a ten-minute maximum lifetime and five-minute rotation interval (Section 3.3.4). This field is the only credential material Saturn broadcasts. Kaiser and Waldvogel [18] established that mDNS-SD exposes all announcement traffic to every machine on the local network. Konings et al. [21] measured this exposure empirically, finding that 59%

of mDNS device names contain personally identifiable information. Saturn’s metadata fields do not contain personal information, but they are visible to any device on the same network segment.

STRIDE Threat Comparison. Saturn’s ephemeral credential model eliminates three high-severity threats present in static API keys (**A5**). *Spoofing*: static keys authenticate the bearer indefinitely; Saturn keys expire in ten minutes, limiting the window for impersonation. *Repudiation*: static keys provide no temporal binding, making it difficult to attribute usage to a specific session; Saturn keys are time-stamped by their rotation schedule. *Elevation of privilege*: a leaked static key grants persistent access to the full API scope; a leaked Saturn key grants access only until the next rotation, at most ten minutes.

The trade-off is three medium-severity threats that require LAN proximity. An attacker on the local network can observe the ephemeral key from mDNS traffic (information disclosure), can use that key for up to ten minutes (tampering), and can register a rogue `_saturn._tcp.local.` service to redirect clients (spoofing at the service level). All three threats require the attacker to be on the same network segment — a constraint that substantially narrows the attack surface compared to internet-facing static keys.

Exposure Window. Meli et al. [23] found that 81% of leaked API keys on GitHub are never revoked after notification, and that the best automated scanning tools detect only 25% of leaks. Their conclusion — that “all mitigations act too late” — describes a fundamental failure of the static key lifecycle: keys persist until someone actively removes them, and evidence shows that removal rarely happens (**A16**).

Saturn sidesteps this failure class entirely. Keys expire automatically after ten minutes regardless of whether anyone detects a leak. The maximum exposure window is

bounded by the key lifetime, not by human response time. A Saturn key leaked to a public repository would be dead before any scanning tool could detect it.

5.3.3 Assessment

Saturn’s security model is a deliberate trade-off, not an oversight. The protocol exposes metadata to the local network — a design property documented by Kaiser and Waldvogel [18] and inherent to any mDNS-based system. Ephemeral credentials mitigate the most severe consequences of key exposure by bounding the damage window to ten minutes. The remaining threats require LAN proximity, which limits them to the same trust boundary that Saturn’s target environments (homes, offices, labs) already assume. **V3** does not claim that Saturn is secure against a determined local attacker; it claims that the trade-offs are known, grounded in existing literature, and addressable through the ephemeral credential lifecycle.

AP Isolation. One deployment constraint directly undermines the campus scenario that motivated this work (**A17**). Enterprise and institutional WiFi networks commonly enable AP isolation, which blocks multicast traffic between wireless clients. I confirmed in January 2026 that both eduroam and UCSC-guest block mDNS. Saturn services advertised on these networks are invisible to client devices. This is not a protocol vulnerability — it is an infrastructure policy decision — but it means Saturn cannot operate on the networks where the access equity motivation is strongest. Saturn works on its target networks (home, office, lab) but fails on institutional networks with client isolation. I return to the implications of this constraint in Chapter 6.

5.4 Cross-Cutting Evaluation: Nielsen Heuristic Analysis

The three claim-specific evaluations above assess what Saturn achieves. A heuristic evaluation assesses how well Saturn communicates its capabilities to its three audiences. I applied Nielsen’s ten usability heuristics [25] to Saturn’s three-persona model (sysadmin, developer, end user) and categorized findings into strengths, trade-offs, and limitations (A18).

5.4.1 Methodology

Nielsen’s heuristic evaluation is an inspection method where an evaluator examines an interface against ten broad usability principles. Unlike the cognitive walkthrough, which traces step-by-step task completion, the heuristic evaluation assesses how well the system communicates across recognized usability dimensions. I conducted the evaluation as a single-author inspection, applying each heuristic to the three Saturn personas.

5.4.2 Strengths

Four heuristics represent clear strengths.

Visibility of system status (H1). Saturn servers appear on the network via mDNS advertisement. Administrators see service registrations through `dns-sd -B` or the router dashboard; developers see available services through `discover()`. End users see nothing — by design, the system works without needing to be seen.

Error prevention (H5). Saturn auto-assigns host addresses, port numbers, priorities, and failover behavior. The system defaults to alternative available services on failure

before raising an error. Configuration slips — mistyped URLs, malformed API keys — cannot occur because the consumer never enters these values.

Recognition rather than recall (H6). The `discover()` function presents available services as a list. No URLs, API keys, or model identifiers need to be recalled. This is the usability mechanism behind **V2**’s step-count reduction: Saturn eliminates the recall-dependent steps that drive traditional setup complexity.

Flexibility and efficiency of use (H7). The Python package provides programmatic control for power users. Traditional API key configuration remains available as an opt-out path: a developer who stops calling `discover()` and supplies their own key bypasses Saturn entirely.

5.4.3 Trade-offs

Three heuristics reveal trade-offs inherent to Saturn’s design.

Match between system and real world (H2). Saturn’s terminology — “beacon,” “proxy,” “deployment” — comes from its internal architecture rather than from the vocabulary sysadmins already use. Developers and end users are less affected: developers see standard OpenAI-compatible endpoints, and end users see nothing.

User control and freedom (H3). Control is role-dependent. Sysadmins start and stop services freely. Developers select specific servers by name through `discover()`. End users have no control over which service they receive; Saturn routes by priority. This asymmetry is intentional — it mirrors the administrator-centric model of network infrastructure — but it means end users cannot override a misconfigured priority.

Consistency and standards (H4). The TXT record schema drift identified in Section 5.1 is also a usability concern. Developers writing clients against the Python core’s schema may encounter services advertised by the Rust router that omit fields the devel-

oper expected. The four core fields are consistent; the optional fields are not.

5.4.4 Limitations

Three heuristics identify gaps that future work should address.

Aesthetic and minimalist design (H8). The schema inconsistency across implementations means that different services advertise different field sets. Clients cannot depend on optional fields being present.

Error recovery (H9). Saturn currently reports “service not available” with no guidance on why discovery failed. No diagnostic mode checks multicast reachability, lists nearby services, or suggests fixes. Error recovery is Saturn’s clearest usability gap.

Help and documentation (H10). Documentation covers sysadmin and developer audiences. End-user documentation is thin because end users are not expected to interact with Saturn directly — but when something goes wrong, they have no resources.

5.5 Threats to Validity

Three categories of threats limit what these evaluations can conclude.

Single-author analytical evaluations. All three claim evaluations and the heuristic evaluation are single-author analytical inspections. A single-author cognitive walkthrough may undercount steps that only surface when a naive user encounters them, because the author’s familiarity with the system can cause them to treat multi-step actions as single steps. The walkthrough scripts mitigate this partially: because each step is documented as executable Python code, an independent reviewer can trace the same sequence and verify the count. The scripts are available in the project repository.

No empirical user data. The cognitive walkthrough counts configuration steps; it does not measure time, error rates, or user satisfaction. A developer who performs four Saturn steps may still spend more wall-clock time than one who performs nineteen traditional steps if the Saturn steps are individually harder. I have no evidence that they are — the Saturn steps (install package, import, call `discover()`, call API) are structurally simpler than the traditional steps (create account, generate key, store key, configure endpoint, implement billing) — but the walkthrough methodology cannot confirm this.

Protocol analysis scope. The security evaluation analyzes what the protocol exposes by design and compares credential lifetimes analytically. It does not include empirical penetration testing, packet capture experiments, or adversarial evaluation. A determined attacker on the local network could exploit the threats identified in Section 5.3 within the ten-minute ephemeral key window. The exposure window analysis bounds the damage but does not demonstrate that the bound is tight.

Scale of the implementation census. Seven consumers across four mDNS libraries establish cross-implementation interoperability, but all seven were written by the same author as part of the same project. Independent implementations by other developers would strengthen **V1** considerably. The census demonstrates that the protocol *can* be consumed independently; it does not demonstrate that external developers *would* adopt it.

Walkthrough persona model. The three-persona model (sysadmin, developer, end user) simplifies real organizational structures. In practice, roles overlap: a developer may also be the sysadmin, or an end user may have some developer skills. The scaling formula assumes clean role separation. Blurred roles would change the per-persona step

counts, though the structural elimination of billing and key management steps would persist regardless of role boundaries.

Chapter 6

Discussion

The introduction framed a specific problem: AI inference costs approach zero, but the access model — individual accounts, API keys, per-user billing — imposes friction that keeps capable services out of reach for users who cannot afford subscriptions or navigate configuration. Saturn proposed that zero-configuration network protocols could eliminate this friction by provisioning AI at the network level. The evaluation confirmed that the protocol works across independent implementations (**V1**), reduces total configuration effort (**V2**), and operates within a documented security surface (**V3**). This chapter examines what those results mean for the problem that motivated the work, how they relate to prior findings, what constraints limit the conclusions, and where the research should go next.

6.1 Implications for AI Access

Saturn’s most immediate implication concerns institutional AI provisioning. Universities currently face a binary choice when providing AI services to students: purchase enterprise subscriptions that bundle unused features at high per-seat cost, or leave stu-

dents to fund their own access. Bassignana et al. [4] documented the consequence of the latter — students from lower socioeconomic backgrounds use AI tools less frequently and benefit less from them. Gabriel [13] observed the same disparity in educational outcomes. Saturn introduces a third option. An institution deploys a Saturn server on its network, funds a token budget with a cloud inference provider, and every device that joins the network discovers the service automatically. Students pay nothing upfront. The institution pays only for tokens consumed, not for seats licensed. No accounts, no API keys, no onboarding friction — the same model universities already use for printers, file shares, and licensed software through network-level provisioning.

This model reframes AI access as a network resource rather than a personal subscription. The cognitive walkthrough (Section 5.2) quantified one consequence of that reframing: end users perform zero configuration steps under Saturn, compared to seven under the traditional model (**A13**). The reduction is not incremental. It represents the complete elimination of the end-user provisioning burden — account creation, payment setup, key management, and application configuration all vanish. Costa et al. [10] identified accessibility and usability as core dimensions of AI democratization; Saturn addresses both by removing the steps that currently gate access.

If Saturn or a protocol like it achieves broad adoption, a second implication follows: application developers could stop treating AI service configuration as their responsibility. Today, every application that consumes AI must implement its own key management, endpoint configuration, and provider selection logic. The walkthrough measured this burden at nineteen steps per application (**A12**). Under Saturn, a developer calls `discover()`, receives a working endpoint, and makes API requests. The four-step developer path (**A12**) assumes only that a Saturn server exists on the network — the same assumption applications already make about DNS and DHCP. Syed et al. [40] documented the current AIaaS landscape of API keys, accounts, and platform lock-

in; Saturn’s beacon architecture sidesteps that entire configuration class by advertising connection parameters at the network layer.

A third implication concerns the barrier to experimentation. The current subscription model requires users to commit money before they can evaluate whether AI services are useful to them. Saturn inverts this sequence. A user on a Saturn-enabled network encounters AI capabilities through applications that discover and consume them automatically. The user need not know that inference is happening, just as a user printing a document need not understand mDNS service resolution. This transparency lowers the threshold for first contact with AI services — users can benefit from AI-powered features in applications they already use without making a purchasing decision or even recognizing that AI is involved. Applications that previously could not justify requiring users to obtain API keys — media players, note-taking tools, accessibility software — gain access to inference through network discovery alone. The VLC integration described in Chapter 4 demonstrates this pattern: a media player that has never been an AI application gains subtitle translation and content summarization through Saturn, with no user-facing configuration.

6.2 Relationship to Prior Work

Saturn’s results extend the zero-configuration networking lineage that Guttman [15] established and that Siljanovski et al. [37] adapted to constrained environments. Those prior efforts demonstrated that mDNS/DNS-SD could provision printers, file shares, and IoT devices without manual configuration. Saturn shows that the same protocols carry AI service metadata — endpoint URLs, authentication tokens, API types, and priority rankings — without protocol extensions or bridging layers. The interoperability evidence (A10) confirms that mDNS/DNS-SD generalizes to a domain its designers did

not anticipate, consistent with Siljanovski et al.’s observation that adopting “preexisting Internet protocols” is preferable to designing new ones.

The security findings align with and extend the mDNS threat models in the literature. Kaiser and Waldvogel [18] documented the passive eavesdropping surface inherent to multicast service discovery; Saturn’s TXT record exposure follows the same pattern they described. Könings et al. [21] found that 59% of mDNS device names contain personally identifiable information, with 32% of users unaware of the exposure. Saturn’s beacon fields — while they do not contain device names — broadcast service metadata to every host on the subnet, creating an analogous information leakage surface. The ephemeral key mechanism (A5) represents a mitigation that prior mDNS systems did not implement: by rotating credentials every five minutes with a ten-minute expiry, Saturn bounds the exposure window in a way that static API keys cannot. Meli et al. [23] found that 81% of leaked API keys are never revoked; Saturn’s automatic expiry removes revocation from the equation entirely.

Saturn’s deliberate divergence from zero-trust principles warrants direct comparison. Ward and Beyer [41] argued that network perimeter security is insufficient and that every request should be authenticated independently of network location. Saturn inverts this philosophy for a specific context: consumer and campus networks where the trust-the-network model already governs resource access. The divergence is not an oversight but a design trade-off (V3). Enterprise environments that require BeyondCorp-level verification are not Saturn’s target deployment — and as Section 6.3 discusses, those environments typically enforce AP isolation policies that block mDNS entirely.

6.3 Limitations

Three constraints bound the conclusions this work can support.

AP isolation blocks institutional deployment. Enterprise and institutional WiFi networks commonly enable AP isolation, which prevents multicast traffic between wireless clients. I confirmed in January 2026 that both eduroam and UCSC-Guest enforce this policy, blocking mDNS discovery entirely (A17). This constraint directly undermines the campus deployment scenario that motivated the project. Saturn functions on home networks, office LANs, and lab environments where multicast traffic flows freely, but it cannot reach the institutional networks where access equity matters most without infrastructure changes outside the protocol’s control. Resolving this limitation requires either institutional network policy changes to permit multicast in designated subnets or a complementary unicast discovery mechanism that operates alongside the mDNS beacon.

Evaluation relies on analytical methods, not empirical user studies. The cognitive walkthrough (A11) and heuristic evaluation (A18) are analytical techniques performed by a single author. They measure structural properties of the interaction — step counts, heuristic compliance — rather than observed user behavior. The 53% step reduction and 79% developer burden reduction are properties of the protocol design, not measurements of actual time saved or errors avoided by real users. Wharton et al. [42] validated the cognitive walkthrough as a method for identifying usability problems in early-stage designs, which matches Saturn’s current maturity. Empirical user studies with diverse participants would strengthen the claims but require a deployment base that does not yet exist (B1).

Saturn is the sole implementation of this pattern. No other system provisions AI services through zero-configuration network discovery (B1). The feasibility claim (V1) rests on one protocol designed and implemented by one author. While the six client implementations and four independent mDNS libraries provide internal diversity, the absence of independent reimplementations means that undiscovered protocol design flaws

may exist. The TXT record schema has already shown signs of drift across implementations (**B6**), with optional fields handled inconsistently between the TypeScript, Rust, and Python clients. Whether the schema stabilizes under real-world use or fragments further remains an open question.

6.4 Future Work

The limitations above point to three concrete research directions.

First, the AP isolation barrier (**B5**) could be addressed through a hybrid discovery architecture that pairs mDNS with a lightweight unicast fallback. A campus deployment could advertise Saturn services via a well-known HTTPS endpoint on the institutional network, allowing clients that cannot receive multicast traffic to discover services through a single DNS lookup. This approach would preserve Saturn’s zero-configuration property for the end user while accommodating networks that block multicast. Designing and evaluating such a hybrid mechanism is the most direct path to the institutional deployment scenario this thesis motivates.

Second, the analytical evaluation methods should be supplemented with empirical user studies. A controlled experiment comparing configuration time and error rates between Saturn and traditional API key provisioning — across participants with varying technical backgrounds — would test whether the step reductions identified in the cognitive walkthrough translate to measurable usability gains. Carmona-Galindo et al. [6] documented how generative AI adoption varies across student populations at Hispanic-serving institutions; a study in that context would directly test Saturn’s potential to reduce the access disparities that Bassignana et al. [4] identified.

Third, Kaiser and Waldvogel’s privacy-preserving mDNS-SD extensions [19] offer a path toward deployments where beacon metadata exposure is unacceptable. Their pri-

vacancy sockets and pairing mechanisms maintain backward compatibility with standard mDNS while encrypting service advertisements to authorized clients. Integrating these extensions into Saturn would enable deployments in environments — such as shared workspaces or medical facilities — where broadcasting AI service availability raises privacy concerns that the current protocol does not address.

6.5 Significance

Saturn demonstrates that the gap between AI’s falling inference costs and its persistent access barriers is a protocol problem, not a cost problem. The technology to provision AI at the network level exists in protocols standardized over a decade ago. What was missing was the application of those protocols to AI service discovery — and the evidence that they work for this purpose without modification. This thesis provides that evidence. Six implementations consume the protocol across three languages. The configuration burden shifts from every user to one administrator. The security surface, while real, is bounded by mechanisms that the mDNS literature already describes. If institutions choose to provision AI the way they provision printers — as a network resource available to anyone who connects — the protocol infrastructure to do so is ready.

Chapter 7

Conclusion

The cost of AI inference is approaching zero, but the cost of *accessing* inference — accounts, API keys, billing relationships — remains a barrier that falls disproportionately on those least able to pay. This thesis asked whether that barrier is necessary. Saturn answers: it is not. Zero-configuration network protocols already provision printers, file shares, and media servers without user intervention; Saturn demonstrates that they can provision AI services the same way.

Three claims structured this argument, and each found support. First, mDNS/DNS-SD can provision AI services without end-user configuration (**V1**). Six independent client implementations — spanning Python, Rust, and TypeScript across four mDNS libraries — consume the `_saturn._tcp.local.` service type with no bridging layer or protocol extensions. A consumer calls `discover()` and receives a working API endpoint. Second, network-level provisioning reduces total configuration effort (**V2**). A cognitive walkthrough across three personas showed a 53% reduction in combined configuration steps, with application developers saving 79% and end users performing zero steps. At modest scale — ten developers and one hundred users — the reduction reaches 94%. Third, the security trade-offs of broadcast-based discovery are documented and

addressable (V3). Saturn deliberately trades enterprise-grade device verification for zero-configuration access, a choice grounded in threat models that Kaiser and Waldvogel [18] established for mDNS and mitigated through ephemeral credentials that expire in ten minutes. The trade-off is deliberate, bounded, and honest about what it gives up.

Saturn does not solve AI access inequity by itself. It solves a protocol problem: how to deliver AI services through the same mechanism that delivers every other network resource. The most pressing constraint is AP isolation — enterprise WiFi blocks multicast between clients, preventing Saturn from reaching the institutional networks where access equity matters most. As Chapter 6 argues, a hybrid unicast fallback or institutional policy change could address this limitation while preserving zero-configuration semantics. Beyond deployment reach, empirical user studies and broader ecosystem adoption remain open directions: Saturn’s value compounds with each application that integrates it, but the protocol’s viability now rests on whether others build on it.

I built Saturn because I was paying \$100 a month for AI services and my peers could not. Universities provision shared resources at the network level every day — a student who joins eduroam gains access to printers and licensed software without creating an account. AI should work the same way. Saturn proves that it can.

Appendix A

Saturn Protocol Specification

This appendix defines the Saturn discovery protocol in sufficient detail for an independent implementor to produce a conforming system. Saturn builds on two IETF standards: Multicast DNS (mDNS, RFC 6762 [8]) for link-local name resolution and DNS-Based Service Discovery (DNS-SD, RFC 6763 [7]) for service advertisement. Familiarity with those RFCs is assumed; this specification covers only the Saturn-specific conventions layered on top of them.

A.1 Service Type

A Saturn service registers under the DNS-SD service type:

```
_saturn._tcp.local.
```

Cheshire and Krochmal [7] define a service type as a pair of DNS labels: an application protocol name prefixed with an underscore and a transport protocol. Saturn uses TCP because all advertised endpoints serve HTTP. Each Saturn instance on the network registers one PTR record mapping `_saturn._tcp.local.` to its instance name,

one SRV record providing its hostname and port, and one TXT record carrying the metadata described in Section A.2.

A.2 TXT Record Schema

RFC 6763 [7] specifies that DNS-SD TXT records carry key-value pairs, each encoded as a single DNS TXT string of at most 255 bytes. Saturn defines the following keys:

Key	Type	Required	Description
version	string	yes	Schema version. Current value: "1.0".
deployment	enum	yes	One of "cloud", "local", or "network". Indicates whether the endpoint is a remote API, a service on the announcing host, or a service shared on the LAN.
api_type	enum	yes	One of "openai" or "ollama". Determines the HTTP API dialect the endpoint speaks.
api_base	URL	conditional	Base URL for API requests (e.g., <code>https://openrouter.ai/api</code>). Required when <code>deployment=cloud</code> ; for local and network deployments, clients construct the base URL from the SRV record's host and port.
priority	integer	yes	Non-negative integer. Lower values indicate higher preference. Mirrors DNS SRV priority semantics (RFC 6763 [7]).
ephemeral_key	string	no	A time-limited API credential. Present only when the beacon rotates credentials on behalf of consumers.
rotation_interval	integer	no	Credential rotation period in seconds. When present, indicates that <code>ephemeral_key</code> will be replaced at

Four keys form the invariant core that every conforming implementation must advertise: `version`, `deployment`, `api_type`, and `priority`. The `api_base` field is required for cloud deployments, where the endpoint URL cannot be inferred from the local network; for local and network deployments, clients construct the endpoint URL from the SRV record's hostname and port. Optional keys refine behavior: `ephemeral_key` supplies authentication for cloud backends, and the remaining keys enable capability-based filtering.

A.3 Required Endpoint Paths

Every Saturn service exposes three HTTP endpoints under its base URL. When `api_type=openai`, the request and response formats follow the OpenAI Chat Completions API convention:

GET `/v1/health` Returns a JSON object with at least a `"status"` field. A 200 response indicates the service is operational. Clients use this endpoint for liveness checks before routing traffic.

GET `/v1/models` Returns a JSON object containing a `"data"` array of model objects, each with at least an `"id"` field. This endpoint provides the authoritative model list, superseding the truncated `models` TXT record.

POST `/v1/chat/completions` Accepts a JSON body with `model` (string), `messages` (array of role/content objects), and optional parameters including `stream` (boolean), `temperature` (float), `max_tokens` (integer), `tools`, and `tool_choice`. When `stream=true`, the response uses server-sent events (SSE). When `stream=false`, the response is a single JSON object.

When `api_type=ollama`, the service translates between the Ollama native API format (`/api/chat`, `/api/tags`, `/api/version`) and the OpenAI-compatible paths

listed above. The translation is the service's responsibility; clients always address the `/v1/` paths regardless of `api_type`.

A.4 Discovery Sequence

A conforming client discovers Saturn services through the following four-step sequence, all of which use standard mDNS/DNS-SD operations defined in RFC 6762 [8] and RFC 6763 [7]:

1. **Browse.** The client issues an mDNS query for PTR records at `_saturn._tcp.local.` on multicast address 224.0.0.251, port 5353. Each PTR response identifies one Saturn service instance by name.
2. **Resolve.** For each instance name returned, the client retrieves the corresponding SRV and TXT records. The SRV record provides the target hostname and port. The TXT record provides the metadata keys defined in Section A.2.
3. **Rank.** The client sorts discovered services by priority in ascending order (lowest value first). Ties may be broken by any client-local policy—the reference implementation uses cost preference ("free" before "paid") as a secondary sort key.
4. **Select.** The client selects the highest-ranked service whose capabilities satisfy its requirements. If the selected service has `deployment=cloud`, the client uses `api_base` as the endpoint URL and `ephemeral_key` as the bearer token. If the deployment is `local` or `network`, the client constructs the endpoint from the SRV hostname and port: `http://{host}:{port}/v1`.

The reference Python implementation introduces a settle-time mechanism: after receiving the first response, the client waits an additional configurable period (default one second) for late-arriving announcements before finalizing the service list. This compensates for mDNS responses that arrive asynchronously across the network.

A.5 Beacon Format and Credential Distribution

A Saturn beacon is a process that advertises a cloud API service on behalf of consumers who lack direct credentials. The beacon does not proxy API traffic—consumers connect directly to the cloud endpoint using the credentials the beacon distributes. This separation is a deliberate architectural decision: the beacon never sees prompts or responses, eliminating it as a data-path bottleneck and a privacy risk.

A beacon operates as follows:

1. **Credential creation.** The beacon authenticates to the upstream provider's provisioning API using the administrator's long-lived API key and requests a short-lived credential. For OpenRouter, this means a POST to `/api/v1/keys` with an expiration timestamp; the response contains a scoped key and a revocation handle. For DeepInfra, a POST to `/v1/scoped-jwt` returns a self-expiring JSON Web Token that requires no explicit revocation.
2. **Advertisement.** The beacon registers on `_saturn._tcp.local.` with a TXT record containing `deployment=cloud`, the provider's `api_base`, and the newly created `ephemeral_key`. The `features` field includes `"ephemeral_auth"` to signal that the key is time-limited and rotated.
3. **Rotation.** At a configurable interval (default five minutes), the beacon creates a new credential, updates the mDNS TXT record by unregistering and re-

registering the service, and revokes the previous credential after a short grace period (default five seconds). The overlap between old and new credentials ensures zero-downtime transitions for active clients.

4. **Shutdown cleanup.** On receiving SIGINT or SIGTERM, the beacon revokes all outstanding credentials and unregisters from mDNS.

A.6 Priority Scheme

The priority field encodes administrator routing policy. It accepts non-negative integers; lower values indicate higher preference. The administrator assigns priorities through TOML configuration files or, on the Saturn router, through the LuCI web interface.

Priorities enable three deployment patterns without any client-side configuration:

- **Local preference.** A local Ollama instance at priority 1 is selected over a cloud OpenRouter beacon at priority 10. Users get the fastest, most private option by default.
- **Automatic failover.** If the priority-1 service becomes unreachable (its `/v1/health` check fails or its mDNS registration disappears), the client falls through to the next available service.
- **Institutional policy.** An IT department sets priorities centrally; users receive the institutionally preferred service without choosing or even knowing alternatives exist.

A.7 Ephemeral Key Lifecycle

Static API keys pose a well-documented management problem at scale. Meli et al. [23] found over 100,000 public GitHub repositories containing leaked API credentials, demonstrating that long-lived secrets inevitably escape their intended scope. Saturn addresses this risk through ephemeral credentials with the following lifecycle parameters:

Parameter	Default Value
Key lifetime (expiration)	10 minutes
Rotation interval	5 minutes
Grace period after rotation	5 seconds
Maximum key length	240 characters (within 255-byte TXT limit)

The five-minute rotation interval and ten-minute lifetime create a five-minute overlap window during which both the current and previous keys are valid. This overlap prevents request failures during the transition. After the grace period, the beacon explicitly revokes the old credential through the provider’s API (or, for providers like DeepInfra that issue self-expiring JWTs, allows natural expiration). On shutdown, all outstanding keys are revoked immediately.

This scheme does not eliminate the exposure window—any device on the local network can read the ephemeral key from the plaintext mDNS announcement. It bounds the window: a captured key becomes useless within minutes rather than persisting indefinitely. Chapter 6 analyzes this trade-off in the context of Saturn’s threat model.

A.8 Conformance Requirements

A conforming Saturn **advertiser** must:

1. Register under service type `_saturn._tcp.local.`
2. Include the four required TXT keys: `version`, `deployment`, `api_type`, `priority`
3. Serve the three HTTP endpoints defined in Section A.3, either directly or by proxying to an upstream that does
4. Unregister from mDNS on shutdown

A conforming Saturn **consumer** must:

1. Browse for `_saturn._tcp.local.` via mDNS
2. Parse the four required TXT keys
3. Respect priority ordering when selecting among multiple services
4. Use `api_base` and `ephemeral_key` when `deployment=cloud`; use SRV host and port otherwise

Optional TXT keys (`models`, `capabilities`, `context`, `cost`, `features`, `rotation_interval`) enhance selection but are not required for basic interoperability. A consumer that ignores all optional keys still connects to and uses Saturn services correctly.

Appendix B

Generative AI in Thesis Development

This appendix will explain the use of LLMs in the completion of this thesis, and during the development of Saturn. This section, along with the Generative AI acknowledgment are entirely hand written me. Every other section of this thesis is the output of a LLM, specifically, Anthropic's Claude series of models. The code for Saturn and the thesis are visible on their respective Github repositories and is entirely the output of these large language models. The methods behind generating this code is described below. These methods are my best attempt to steer LLMs to be more reflective of my true intentions, design patterns, and beliefs. This system provided me with an agent that could serve as an agentic advisor, going beyond the guidance of one hour weekly sessions with the primary research advisor. This agentic workflow is effective at designing a thesis, retrieving facts and claims from memory, and correcting itself. This system hopes to provide a level of sophistication for papers that use generative AI in their creation.

B.1 Design Fictions

Design fictions are an existing practice of creating fictional scenarios where a non-existing idea becomes real, with the goal of creating speculative and provocative discussions about the idea. Design fictions have been used in debate for decades, however they provide a unique advantage with LLMs to create specification documents for a system. Design fictions provide a working example of an idea that can be reverse engineered by an LLM. The design fictions for Saturn can be found in the fiction directory of the Saturn Github repository. They follow the story of two fictional characters, Derek and Mira, and a fictional corporation: Megalink. Derek's role was a software engineer and self-proclaimed "home network wizard," meant to reflect a technically proficient integrator of Saturn who has configured Saturn on their network. The Megalink corporation is the company that created the system that would ultimately become Saturn. Mira symbolizes a non-technical user. She is the sister of Derick who visits and uses Saturn on Derick's network to generate AI captions on a fictional photo app. She didn't need to pay for anything or configure settings, she just joined Derek's network and she had new features that were not present before. Later, a new character, Jordan, was created to inspire the beacon system. Jordan was a developer who had leaked his company's API key from a Saturn server. Beacons were created to provision short lived keys on the network that are deleted after a short window. As said before, when a LLM reads this design fiction, it isn't interpreted as speculation, it instead becomes a vision that can be broken down into specs. The three characters provide the LLM with examples of who in the real world would use Saturn. There is an understanding that the end user (Mira) and the programmer who configures Saturn on their network (Derek) require different types of documentation and tools. These design fictions lead to the initial specifications for the Saturn protocol and first implementations.

B.2 Documentation Website

During development of Saturn, a human and agent facing documentation website was created with the intent to be referenced to a coding agent in future context. Myself, or any other person could visit the website and use it as a reference for their coding agents, as it served as the main knowledgebase for Saturn. In fact, the documentation site has a `llms.txt` file specifically designed to guide AI agents. The documentation is split across three different sections: the user guide, the integrator guide, and an integrations page. The user page provides a high level overview of the system, and a short guide for discovering Saturn services on the network. I purposefully tried to avoid technical jargon such as mDNS, TXT records, and endpoints. This page can therefore serve as an introduction to new users of the system, or it can be injected into the prompt for an agent for context about Saturn. The integrator guide provides the technical overview of Saturn and is meant to be read by developers and network administrators. This page includes how to configure a custom Saturn service, which endpoints were required, and Saturn SDK guides. This page provided basic context to agents implementing Saturn. The last page showcased integrations of Saturn in hopes of growing popularity for the protocol. If I designed a page that could prove how easy it is to implement Saturn as a protocol, it could become more popular and therefore standardized. Coding agents could be fed this site as context to understand how similar platforms have integrated Saturn in the past. Integrators or coding agents could submit their personal integrations of Saturn on a form on this page, which then become Github Issues for the website. This website was frequently used as a source of knowledge that was fed to an agent upon fresh work sessions. Without it, Saturn would need to be explained every session, risking the loss of implementation details and worse context for the agent. This webpage later served as an initial source of memory for the Moons system, which will be described later in

the appendix.

B.3 Integrations into Various Applications

An important goal of Saturn was to create multiple interoperable technical prototypes built around the protocol `_saturn._tcp.local`, demonstrating that widespread adoption was feasible. Initially Saturn was integrated into the chat based AI applications OpenWebUI and Jan, later moving to the coding agent OpenCode. All these platforms were natively based in AI, meaning some readers or developers may disregard Saturn entirely if they are working on projects or apps that are not AI native. Adam and Joey believed that Saturn made AI possible in apps that were not AI native, and to prove this, Joey created a VLC extension that roasted a user based off of what media was playing. There was no chat, and no mention of AI whatsoever. Developing these integrations could have each been a master project on their own. However, Joey took advantage of coding agents to develop them faster instead. I was able to integrate to these platforms faster because they would often have thorough documentation or source code on the web to reference to a coding agent. This had a snowball effect where the more integrations got built, the more examples of Saturn could be given as context to future agent sessions.

B.4 Serve Role as a “Robotic” Advisor

Joey and Adam met once a week on an hourly basis to discuss Saturn. Outside of that hour, the AskUserQuestion tool incorporated into Claude Code would assist in taking notes for writing the thesis paper. I actively engaged with Claude Code to arrive at different claims for the thesis, organize background work, and create an evaluation

plan by chatting, and answering questions. Initially, the agent wrote all of its memory of Saturn and the interviews in a single markdown file. This file was disorganized, lacked technical depth, or misinterpreted information I would reference. The agent struggled to remember previous conversations the student would have, and it would have difficulty piecing together the repository, previous academic work, and the claims of the system. There needed to be a system that gave agents the same level of knowledge I had about Saturn, while providing the ability to hyperfocus on specific sections of the thesis depending on the focus of a work session.

B.5 Moons

Moons is the memory system that served as the knowledge base for all coding agents while I wrote the thesis. Moons did not exist while Saturn was being developed, only during the writing process of my thesis. The name “moons” comes from Saturn, the planet, notoriously having a lot of moons. Each individual moon can be thought of containing one small piece of information about the greater and more complex Saturn. Moons is a JSON file that points to different directories and markdown files in the moons directory. Each entry in this JSON file has the following fields:

id: the element of saturn that the current chunk is describing. Example: claim1, beacons, ai-sdk

type: this describes what role the id has in the saturn project. Example: claim1 has a type of claim, and beacons has a type of concept.

Desc: A short one sentence description of what the element is. Example: ephemeral-keys description reads “Time-limited API credentials (10-min lifetime, 5-min rotation with overlap for zero-downtime transitions) distributed via mDNS TXT records and deleted on shutdown, paralleling Kerberos session-layer distribution. Implemented in

Python (KeyManager with thread-safe locks) and Rust (full lifecycle with ‘Drop’ impl), making the ‘leaked secret never removed’ class (Meli 2019: 81%) structurally impossible.”

File: reference to where a detailed markdown file of the concept is. These files contain a more detailed explanation of the ID, where to find the source files related to the ID, and why it matters in the project.

Edges: list of other ids and their relation to the current chunk

Example moon chunk:

```
{
  "id": "mdns",
  "type": "concept",
  "desc": "Link-local name resolution (RFC 6762) via multicast to 224.0.0.251:5353 requiring no DNS server | Saturn's protocol foundation, used identically to how Bonjour discovers printers, AirPlay, and Chromecast. Key limitations: AP isolation blocks multicast (confirmed on eduroam/UCSC-guest Jan 2026), passive eavesdropping exposes all announcements, and 59% of device names leak real user names.",
  "file": "concepts/mdns.md",
  "edges": [
    {
      "to": "claim-1",
      "rel": "supports"
    },
    {
      "to": "dns-sd",
      "rel": "related"
    },
    {
      "to": "ch2",
      "rel": "discussed_in"
    },
    {
      "to": "ch3",
      "rel": "discussed_in"
    }
  ]
},
```

The goal of moons is to create a knowledge graph for an agent. With moons, a conversation about Saturn is much easier because I could freely reference source material with my agent having a memory bank to reference. An example question would be: “What have we discussed in claim 2 so far, is there anything I am forgetting to add?” An agent can read the graph.json to find claim-2’s id and trace all the details for claim-2

through markdown files. I, the human in the loop, could see if my agent is forgetting any information, or hallucinating the results and claims I previously made. A drawback to this strategy is that the graph.json has to be read in its entirety every time moons are mentioned. This drawback helps provide Claude with a high level understanding of all of the components of Saturn and how they all connect, but ultimately fills the context window with irrelevant information. My alternative option was to query the graph via a script, but this strategy proved to be even more inefficient as one read call is cheaper than multiple bash calls considering Saturn's graph.json was ~563 lines. The agent also has to have knowledge of the moons system each session, and the only way to effectively provide the knowledge of moons existence is via an AGENTS.md file or a CLAUDE.md file. This ended up taking 20 lines in my CLAUDE.md file which is already a context burden for most systems, but proved to not have a noticeable effect for this thesis.

Moons (Knowledge Base)

This project uses a structured knowledge graph called 'moons/' to organize all thesis context.

****Structure:****

- 'moons/graph.json' | the map. Lightweight nodes with typed edges. Read this FIRST to navigate.
- 'moons/concepts/' | Saturn concepts (mDNS, DHCP, ephemeral keys, etc.)
- 'moons/claims/' | the 3 thesis claims with evidence
- 'moons/papers/' | distilled paper summaries with relevance to Saturn
- 'moons/voice/' | user's personal perspective, design rationale, interview notes

- 'moons/code/' | code component analyses with line citations
- 'moons/chapters/' | chapter planning and section status
- 'moons/meta/' | advisor notes, timeline, project context

****How to use:****

1. Always 'Read(moons/graph.json)' first. At ~500 lines it's cheap to read in full and gives complete graph visibility | all nodes, edges, and types in one shot. This holistic view is intentional: it surfaces connections you weren't looking for.
2. Follow edges to specific content files only when you need depth.
3. After interviews or new information, update both the relevant content file AND graph.json edges.
4. Keep graph.json lean | short descriptions, not full content. Depth lives in the files.
5. 'moons/query.py' exists as a human CLI convenience ('python moons/query.py edges-to claim-3').

****Moons is the workspace. thesis.tex is the submission.**** Moons organizes thinking; thesis.tex is the final output.

I often discovered gaps of knowledge or evidence in Saturn by viewing the moons knowledgebase. I would notice certain chunks had less edges than I believed they should, leading me to discover I forgot to upload an implementation piece of Saturn to the thesis. Moons highlighted areas of Saturn that had the most evidence, and which areas needed more support or focus. Coding agents also had knowledge of how moons worked, meaning they could create interactive websites for visualizing Saturn's ideas.

B.6 Finding Academic Papers

Google announced Scholar Labs during the research phase of Saturn. Google Labs is a search engine that is powered by Gemini, and is developed like most chat bots; a user can query in natural language and the model will find academic papers related to that question. With the knowledge stored in moons, I could ask my agent to generate research questions for my thesis to be later sent to Scholar Labs. Once I had a result I would read the abstract, results, and figures of each paper before downloading and storing them inside the moon's knowledge base. The agent would always require approval for its integration of the paper, and this process was continuously iterated for finding more papers. The moon knowledge base grew as my own understanding and research behind Saturn grew.

B.7 AI Contribution in the Evaluation Section

The second claim in the evaluation section of this thesis was that Saturn ultimately provided less cognitive overhead than traditional API configuration methods. To prove this, my strategy was to perform a cognitive walkthrough, where I documented a series of steps in order to convey the workload required to accomplish each configuration setup. The initial goal was for an agent to autonomously count all the steps with the MCP server Playwright in isolated Docker containers. This ended up failing as the system was unreliable and difficult for me to verify. Perhaps with the invention of OpenClaw I overlooked an entirely agentic system. Instead, I opted for a different approach that relied on my previous design fiction's three different audiences for Saturn. There are people who configure Saturn on the network, app developers incorporating Saturn, and end users who consume the ai services. I first performed a 100% human-

written cognitive walkthrough of creating an API key on Openrouter and placing it in an environment variable file. These were shared steps that both a system with and without Saturn needed to take. After this initial cognitive walkthrough, I wrote python scripts from the design fiction’s multiple perspectives:

- An app developer using saturn’s python package to send a query to an api endpoint
- An app developer using the traditional method of querying an api endpoint (including Stripe billing)
- The cognitive walkthrough for a sys admin distributing api keys to their team
- The cognitive walkthrough of a sys admin configuring a saturn beacon on their network without using the saturn python package
- The cognitive walkthrough of a sys admin configuring a saturn beacon on their network using the saturn python package
- An end user using an app with saturn on their network
- An end user using an app that uses AI without saturn

Note: These python scripts are as minimal as possible and try to avoid excessive features. The end user scripts contain no runnable code, rather just a docstring of the walkthrough this fictional person went through.

B.8 Academic Writing Skill

LLMs played a pivotal role in generating the text written in the thesis document. I created Moons to solve the problem of giving the agent memory of Saturn, and I uti-

lized the AskUserQuestion tool to conduct self-guided interviews on my thoughts and claims for the thesis. Still, I faced a consistent problem: AI agents continuously failed to write what I considered to be captivating and well-written academic papers. The papers themselves are long and flood context, not including the knowledge required to remember what arguments and evidence are required when writing the paper. Even with the moon's knowledge base, it was impossible to one-shot a well written thesis in a singular prompt. Dividing the thesis into context-manageable sections wasn't effective either as the agent would often repeat information referenced in other sections of the thesis. I also believe AI writing to be subpar and lacking substance, with most of its output content following the same structure that eventually becomes a glaring pattern. Additionally, models had the tendency to scatter knowledge with no cohesion. To solve this problem I created the academic-writing skill. An agent will invoke this skill when the user directly mentions it or the description matches the problem the user is asking the agent to solve. For the academic-writing skill my description read:

“Use this skill whenever writing prose, essays, academic content, reports, or any long-form written output, even if the user doesn't explicitly mention writing quality. Trigger for any drafting, revising, or editing of written sections — including when the user pastes text and asks for feedback.”

The skill's body includes general writing skills that I personally believed to be best practice and reflected how I wanted the thesis to be written. The rules of writing were divided into the following sections: Concreteness, Reader-Centered Writing, Dialogic Framing, Structure, and Rigor. The body also included a core workflow of drafting, evaluating, revising, and presenting the revision. These rules and the workflow were written to be generalizable enough that they could be applied to any area of my thesis. I wanted individual sections of my thesis to have specific rules and guidelines for writing,

so markdown files were created for rules on that specific section of the thesis. The main skill body directed the agent to read the markdown file associated with the section it was writing. Now, agents could invoke this skill to write or make changes to specific sections (like the abstract or evaluation) of the thesis. It had the rules to generally follow, but also the rules for writing specific areas. There is no mention of Saturn itself in this skill, as I wanted the agents to understand these rules applied to all types of academic writing and not Saturn in isolation.

B.9 AI Detection Ralph Loop

Ralph loops were invented by Geoffrey Huntley and they allow a user to run fresh agent sessions with the same prompt infinitely. A simple Ralph loop looks like this:

```
while ;; do cat PROMPT.md | claude-code ; done
```

This trick allows agents to correct themselves and complete a long repetitive task autonomously. While I was writing my thesis, I noticed that no matter how many interviews or evidence there was in moons, the agent would still write vapid claims or fail to see the thesis as a whole piece of work, not individual sections. Initially, I planned to run Sapling, an ai detection software, inside a Ralph loop so an agent can receive a score and try to minimize the score for the next pass. Adam was strongly opposed to this idea, as there was no clear goal or structure behind beating the score. An agent could essentially game the system and not actually improve the quality of writing. So, Adam and Joey read through sections of the thesis and transcribed to an agent what they believed to be common mistakes and blunders the agent was making when writing. Claude then organized this transcription into “academic-advisor-notes.md.” These notes served as guidelines that a subagent was given to grade my thesis. After

this session, I designed a new Ralph loop. The Ralph loop begins with an agent reading notes from the previous Ralph session, the academic advisor notes, and the moons graph. Then, this agent spawns a subagent that reads the content of the thesis and grades based on the guidelines of Adam and I. This read-only subagent returns the grade back to the main agent with specific sections annotated as needing more correction than others. The main agent uses the knowledge it previously read about, and the academic-writing skill, to rewrite the thesis. After the thesis is finished, the agent takes notes of the score the thesis received before edits and the changes it took to write those sections better. These notes have the goal of steering future agents towards some writing strategies and away from others. If the previous Ralph iteration made major changes but the score decreases in the next session, that future agent knows to avoid the tips that the old agent left notes for. This Ralph loop process ultimately led to the creation of the first thesis draft that I eventually tweaked section by section.

B.10 Structured Arguments Skill

The structured arguments skill was a skill used for an agent to organize my claims, evidence, reasoning, arguments, etc in a format that was modeled after Belcak et al. (2025) “Small Language Models are the Future of Agentic AI.” This paper labels every claim, argument, and counter-argument with short IDs and cross-references them throughout. This was a formatting skill but ultimately made the thesis easier to follow while eliminating my burden of manually labelling.

Bibliography

- [1] Anthropic. Claude pricing, 2025.
- [2] Anysphere. Cursor: The AI Code Editor, 2025.
- [3] Apple Inc. Bonjour, 2002.
- [4] Elisa Bassignana, Amanda Cercas Curry, and Dirk Hovy. The ai gap: How socioeconomic status affects language technology interactions. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 18647–18664, 2025.
- [5] BerriAI. Litellm: Call 100+ llm apis using the openai format, 2025.
- [6] Víctor D. Carmona-Galindo, Hou Ung, Manhao Zeng, Christine Broussard, Elizaveta Taranenko, Yousef Daneshbod, David Chappell, and Todd Lorenz. Generative ai as a sociotechnical challenge: Inclusive teaching strategies at a hispanic-serving institution. *Knowledge*, 5(3):18, 2025.
- [7] S. Cheshire and M. Krochmal. DNS-Based Service Discovery. RFC 6763, February 2013.
- [8] S. Cheshire and M. Krochmal. Multicast DNS. RFC 6762, February 2013.
- [9] Continue. Continue: Open-Source AI Code Assistant, 2025.
- [10] Carlos J. Costa, Manuela Aparicio, Sofia Aparicio, and Joao Tiago Aparicio. The democratization of artificial intelligence: Theoretical framework. *Applied Sciences*, 14(18):8236, 2024.
- [11] Digital Living Network Alliance. DLNA Guidelines, 2017. DLNA merged with SpireSpark International in 2017.
- [12] R. Droms. Dynamic Host Configuration Protocol. RFC 2131, March 1997.
- [13] Sonja Gabriel. Generative ai and educational (in)equity. In *Proceedings of the 4th International Conference on AI Research (ICAIR 2024)*, pages 133–142, 2024.

- [14] NetBIOS Working Group, Defense Advanced Research Projects Agency, Internet Activities Board, and End to End Services Task Force. Protocol standard for a netbios service on a tcp/udp transport: Concepts and methods. RFC 1001, March 1987.
- [15] Erik Guttman. Autoconfiguration for ip networking: Enabling local communication. *IEEE Internet Computing*, 5(3):81–86, 2001.
- [16] Jan.ai. Jan: Open-source chatgpt alternative, 2025.
- [17] Jordi Bruin. MacWhisper: Transcription Using OpenAI’s Whisper, 2025.
- [18] Daniel Kaiser and Marcel Waldvogel. Adding privacy to multicast dns service discovery. In *IEEE 13th International Conference on Trust, Security and Privacy in Computing and Communications*, pages 809–816, 2014.
- [19] Daniel Kaiser and Marcel Waldvogel. Efficient privacy preserving multicast dns service discovery. In *IEEE International Conference on High Performance Computing and Communications (HPCC)*, pages 1261–1268, 2014.
- [20] Tae Hyun Kim and Douglas Reeves. A survey of domain name system vulnerabilities and attacks. *Journal of Surveillance, Security and Safety*, 1:34–60, 2020.
- [21] Bastian Könings, Christoph Bachmaier, Florian Schaub, and Michael Weber. Device names in the wild: Investigating privacy risks of zero configuration networking. In *IEEE 14th International Conference on Mobile Data Management*, pages 51–56, 2013.
- [22] LibreChat. LibreChat: Enhanced ChatGPT Clone, 2025.
- [23] Michael Meli, Matthew R. McNiece, and Bradley Reaves. How bad can it git? characterizing secret leakage in public github repositories. In *Proceedings of the 2019 Network and Distributed System Security Symposium (NDSS)*, 2019.
- [24] Microsoft. Visual Studio Code Remote Development, 2025.
- [25] Jakob Nielsen. 10 usability heuristics for user interface design. *Nielsen Norman Group*, 1994. Updated 2024.
- [26] OASIS. Web Services Dynamic Discovery (WS-Discovery) Version 1.1. Technical report, OASIS, July 2009.
- [27] Ollama. Ollama: Run Large Language Models Locally, 2025.
- [28] Open Connectivity Foundation. UPnP Device Architecture 2.0. Technical report, Open Connectivity Foundation, 2015.

- [29] Open WebUI. Open webui, 2025.
- [30] OpenAI. Chatgpt plus, 2025.
- [31] OpenAI. Openai api pricing, 2025.
- [32] OpenRouter. Openrouter: A unified interface for llms, 2025.
- [33] Lennart Poettering et al. Avahi: A Free Zero-Configuration Networking (Zero-conf) Implementation, 2004.
- [34] Emad ul Haq Qazi. Api security: Challenges and best practices. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, 9(4):198–205, 2023.
- [35] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust Speech Recognition via Large-Scale Weak Supervision, 2022.
- [36] Farhan Siddiqui, Sherali Zeadally, Thabet Kacem, and Scott Fowler. Zero configuration networking: Implementation, performance, and security. *Computers and Electrical Engineering*, 38(5):1129–1145, 2012.
- [37] Aleksandar Siljanovski, Anuj Sehgal, and Jürgen Schönwälder. Service discovery in resource constrained networks using multicast dns. In *Proceedings of the IEEE International Conference*, 2014.
- [38] Lewis L. Strauss. Speech to the national association of science writers, 1954. New York City, September 16, 1954.
- [39] SuperWhisper. SuperWhisper: AI Voice to Text for macOS, 2025.
- [40] Naeem Syed, Adnan Anwar, Zubair Baig, and Sherali Zeadally. Artificial intelligence as a service (aiaas) for cloud, fog and the edge: State-of-the-art practices. *ACM Computing Surveys*, 57(8):211:1–211:36, 2025.
- [41] Rory Ward and Betsy Beyer. Beyondcorp: A new approach to enterprise security. *login*., 39(6):6–11, 2014.
- [42] Cathleen Wharton, John Rieman, Clayton Lewis, and Peter Polson. The cognitive walkthrough method: A practitioner’s guide. In Jakob Nielsen and Robert L. Mack, editors, *Usability Inspection Methods*, pages 105–140. John Wiley & Sons, 1994.
- [43] Wispr. Wispr Flow: AI Dictation That Writes How You Speak, 2025.